LLIFS Specification

Draft v1.0

Contents

LLIFS Specification

Status: Draft (v1.0-draft). All sections (§§1-17) drafted, section-reviewed, reconciled end-to-end (terminology normalized, requirement IDs collision-free, encodings byte-exact), and revised against two external technical reviews: the first (memory byte model, proof-block taxonomy, delta self-containment, workload-identity breadth, yield semantics, local-CAS integrity, directfs, inline reserved in v1, the Phase-1 density-gate ladder); the second adding the per-agent runtime-state delta (DELTA-2R / RDELTA / LLRD1), virtual zero-subtree proof rules (SPARSE-8), startup-cohesion reserved in v1, an exact oci_image_digest form (ENC-BD-4), LLMD1 source-precedence (ENC-MD-5), a named compatibility verifier step (MATRIX-3), and the gVisor page-provider distinction (GVISOR-6). A third hardening pass then defined the RDELTA-1 no-runtime-state exception (a signed base-descriptor field, ENC-BD-5, whose semantics are a resume-time guarantee for any later memory-persisting snapshot, not merely a property of the base restore point) and its verifier step, fixed the opaque-runtime-object framing convention (ENC-3: base runtime_state and the runtime delta are raw, unframed; all other encodings are framed), forbade the runtime delta from carrying memory/filesystem bytes (ENC-RD-1), pinned the virtual zero-subtree derivation recurrence (SPARSE-8), split runtime-delta observability (OBS-9), and added the four-interface Phase-1 implementation surface (§16.1). Phase 1 prototyping is green-lit from this draft. The §15.3 Terrapin vectors are now CROSS-CONFIRMED by two independent v0.3 oracles (terrapin-rs and terrapin-go), closing ENC-CONF-2; per-level zero-subtree roots (SPARSE-8) are pinned. HARD FREEZE BLOCKERS (v1.0 MUST NOT freeze until done): runtime-delta implementation proof on a real gVisor hibernate/resume path; and reconcile the §15.8 LLWI1 field set against atelet.WorkloadSpec (ENC-WI-1) - a workload-identity mismatch is a restore-safety bug.
System: LLIFS (a verified, deduped, lazily-faulted state substrate for high-density agent FaaS).
Primary target: gVisor (runsc) checkpoint/restore; Firecracker microVM next. One content-addressed, Terrapin-verified store delivers two state planes: a filesystem plane and a memory plane.
External identity: OCI-compatible sha256.
Internal identity: terrapin-sha256, profile llifs-terrapin-sha256-v2.
Dependency: Terrapin Specification v0.3 (the hashing primitive). Its profile parameters and the load-bearing conformance constants are inlined here (§2.2, §15); the algorithm itself is normative-by-reference.

Introduction

LLIFS is the storage and state-management layer for running very large numbers of sandboxed agent workloads on a single fleet. It exists to make two things simultaneously true: a sandbox starts in roughly the time it takes to fault in the few blocks it actually touches, and every byte the sandbox is ever shown has been cryptographically verified against a trusted identity before execution sees it. Fast start and verified state are not traded off against each other; the design delivers both at once.

The setting is agent FaaS: thousands to millions of agent instances that are near-identical, the same base image and the same warmed runtime (interpreter, framework, model client), differing only in a small amount of per-agent state. Treating each instance as an independent copy of a full filesystem and a full memory image does not scale. It is too slow to start, because you move whole images before the first useful instruction runs, and too expensive to keep, because you store the same bytes a million times. The core thesis follows from this: do not optimize "pull the whole state faster." Start the workload after fetching only the bytes required for useful execution, while verifying every

exposed byte, and make the shared part genuinely shared so that a million near-identical agents collapse to one verified base plus a small per-agent delta.

The approach. LLIFS keeps exactly one verified, content-addressed copy of each distinct byte-range and composes every sandbox from a shared base plus a small private overlay:

- Two state planes. A filesystem plane (the rootfs the workload reads and writes) and a memory plane (a warmed guest-memory image a sandbox restores from). Both are built from one content-addressed store.
- Lazy, verified fault-in. Bytes are fetched and verified in fixed blocks on demand, so a workload begins as soon as the blocks on its startup path are present, not after a whole image transfers. Nothing reaches the workload until its containing block has verified (§8).
- Shared copy-on-write base. One base filesystem and one base memory snapshot back many sandboxes; only the files and pages an agent actually changes become private (§4.5, §5, §6).
- Content-addressed identity end to end. Every internal object is named by its content (§2), so identical content is automatically a single object; this is what collapses near-identical agents into one base plus deltas.
- A per-snapshot State Root. The complete, restorable identity of one agent snapshot (its base, its memory delta, its filesystem delta, its runtime delta, and the runtime it is pinned to) is itself a single content-addressed value with a parent pointer, giving a lineage DAG and cheap forking (§6).

Two identity planes. LLIFS speaks two digest "languages" and never confuses them (§2). The external plane is OCI-compatible sha256, so images, registries, scanners, signatures, and admission policy interoperate unchanged. The internal plane is terrapin-sha256 (profile llifs-terrapin-sha256-v2): the verified, length-bound, tree-structured identity over which all fetch, dedup, and copy-on-write decisions are made. Terrapin itself, the hashing primitive, is a separate dependency; this document inlines only its profile parameters and load-bearing constants (§2.2, §15) and references the algorithm normatively.

Primary target. The first runtime is gVisor (runsc) checkpoint/restore, with Firecracker microVMs as the next adapter (§11). The design reuses existing runtime restore machinery wherever it can; the one capability it needs that does not exist yet (a shared copy-on-write memory base) is stated as a runtime requirement (§5, §11).

What this document is. This is the single normative specification for LLIFS. It defines the object model and identity scheme (§2), the two state planes (§§3–5), the agent lifecycle (snapshot, reset, hibernate, yield, resume, and fork) (§§6–7), the distribution, caching, and garbage-collection model (§§8–10), the control-plane binding (§12), the security model (§14), and, so that independent implementations agree byte-for-byte, the canonical on-the-wire encodings and conformance vectors (§15). It consolidates a family of earlier design documents into one; all cross-references are internal section references.

What this document is not. It does not define the Terrapin hashing algorithm (a referenced dependency), a scheduler or registry implementation, or any deployment-specific policy. Items the operator must supply are called out as out of scope (§14.4). The requirement keywords are normative per §1.

How to read it. Read this introduction, then §1 (normative language) and §2 (identity and the canonical vocabulary). §2 and the glossary (§17) are the terminology anchor for the whole document: every other section uses those terms and no synonyms. From there, §§3–5 define the filesystem and memory planes; §§6–7 the agent lifecycle, State Root, fork, and resume; §§8–10 fetch, verification, caching, GC, and scheduling; §§11–13 runtime adapters, control-plane binding, and observability; §14 the security model; §15 the canonical encodings and conformance vectors (the conformance surface for an implementation); §16 the phasing and minimum viable surface; and §17 the glossary.

Document outline

```
  4.  Filesystem plane: content-addressed layout (LLAY1)        [complete]
  5.  Memory plane: base snapshot, copy-on-write restore        [complete]
  6.  Agent state and lifecycle: overlay, delta, State Root, lineage, fork, reset/hibernate/yield   [complete]
  7.  Startup and resume: profiles, packs, bootstrap, TTFE/TTR  [complete]
  8.  Fetch, verification, transport                            [complete]
  9.  Cache, cache domains, leases, garbage collection          [complete]
  10. Scheduler integration and warmth                          [complete]
  11. Runtime adapters (gVisor, Firecracker)                    [complete]
  12. Control-plane binding                                     [complete]
  13. Observability                                             [complete]
  14. Security model                                            [complete]
  15. Canonical encodings and conformance vectors              [complete]
  16. Phase plan and minimal viable surface                     [complete]
  17. Glossary                                [complete]

------------------------------------------------------------
```

1. Normative language

```
The keywords MUST, MUST NOT, REQUIRED, SHALL, SHALL NOT, SHOULD, SHOULD NOT,
RECOMMENDED, MAY, and OPTIONAL are to be interpreted as normative requirements.

------------------------------------------------------------
```

2. Identity and digest model

```
LLIFS has two digest planes and one canonical object vocabulary. This section is
the terminology anchor for the whole document: every other section uses the
canonical terms defined in §2.3 and §17, and no synonyms.

------------------------------------------------------------
```

2.1 Two digest planes

```
External digest plane: for compatibility with OCI registries, descriptors,
image manifests/indexes, config and layer blobs, OCI artifact descriptors, scanners,
SBOMs, signatures, and admission policy. The external digest algorithm is
sha256.

Internal digest plane: for LLIFS-native identity and verification. The internal
digest algorithm is terrapin-sha256, profile llifs-terrapin-sha256-v2. It is the
identity of every internal object: content objects, base memory snapshots, base
descriptors, deltas, State Roots, the logical rootfs tree and physical layout
encodings, and the runtime-state blob (startup cohesion is reserved in v1, §4.4).
(Hash-file/proof blocks are not separate objects: they are the level >= 1 blocks of
an owning object, verified through its tree, TERRAPIN-6.) Local cache keys (§9) are
DERIVED from these identifiers (directly, or HMAC-keyed per cache domain); they
are not themselves objects.

There is exactly one internal identifier scheme: terrapin-sha256:<64 lowercase
hex>. There is no separate per-object-class digest scheme; objects differ by
their content and their declared class, not by their hash namespace.

DIGEST-1:
  External OCI object identity MUST use sha256.
DIGEST-2:
  Internal LLIFS object identity MUST use terrapin-sha256 (profile
  llifs-terrapin-sha256-v2). The bare recursive tree root MUST NOT be used as an
  identifier (it is not self-describing about length or tree height).
DIGEST-3:
  External and internal identifiers MUST NOT be substituted for one another. The
  OCI digest proves registry object identity; the Terrapin digest proves LLIFS
  content identity (§2.5).

------------------------------------------------------------
```

2.2 Terrapin profile (llifs-terrapin-sha256-v2)

```
Parameters (fixed for this profile):

  algorithm:   terrapin-sha256
  block size:  2097152 bytes exactly (2 MiB; NOT 2,000,000)
  leaf hash:   gitoid-sha256
  node hash:   gitoid-sha256
  identifier:  G(canonical Terrapin root manifest)
  digest form: terrapin-sha256:<64 lowercase hex>

GitOID SHA-256:

  G(data) = sha256("blob " || decimal(len(data)) || "\0" || data)

where decimal(n) is base-10 ASCII with no leading zeros and "\0" is one NUL byte.
G(data) is 32 bytes. The "blob <len>\0" framing binds input length into the
digest, detecting truncation and short writes at every level.

The Terrapin identifier is G over a canonical four-field ASCII manifest
(terrapin, block_size, length, tree), LF-terminated, in that fixed order. A
manifest that does not parse to exactly the canonical form MUST be rejected, not
normalized. (Full manifest grammar and accept/reject matrix: §15.)

Load-bearing constants (the rest of the vector set is in §15):

  G(empty)                         = 473a0f4c3be8a93681a267e3b1e9a7dcda1185436fe141f7749120a303721813
  G(2 MiB zero block) [leaf]       = 67cbed9b97ddabde2863f4daefa4f57176567a7c3ccfa1560c1065f9c8af74d6
  identifier("hello world", len 11)= terrapin-sha256:7bc0163f32e5f6082308ae0dff3dc7c9b0488e5aa652d9de01418df5ec800c8c

TERRAPIN-1:
  New LLIFS objects MUST use profile llifs-terrapin-sha256-v2.
TERRAPIN-2:
  imagefsd MUST verify, for every object, that G(canonical Terrapin manifest)
  equals the object's terrapin-sha256 identifier.
TERRAPIN-3:
  The fixed 2 MiB block is the unit of fetch and verification (a "block", §2.3).
  No sub-block proofs and no sub-2-MiB profile are defined; finer granularity, if
  ever needed, is a separate profile justified by measured over-fetch, never by a
  desire for finer dedup (dedup is achieved structurally: §2.3, §4, §5).
TERRAPIN-4:
  Bytes MUST NOT be exposed to a workload until the containing block has verified
  against a trusted Terrapin identifier. Peers, mirrors, registries, caches, and
  packs are transport, never trust anchors.
TERRAPIN-5:
  A content object of size <= 2 MiB has no hash-file/proof blocks: the data block
  is the leaf and the tree root is G(data). Only multi-block objects carry
  level >= 1 hash-file blocks; one 2 MiB level-1 block authenticates up to a
  128 GiB contiguous region.
TERRAPIN-6:
  Hash-file/proof blocks are NOT independent objects: they are the level >= 1
  blocks of an owning Terrapin object, addressed by BlockRef (object / level /
  blockIndex, §2.3 block) and verified through that object's manifest and tree.
  Only the object classes of §2.3 (and the structural plane objects of §2.3 intro)
  are Terrapin objects with their own identifier; data blocks (level 0) and proof
  blocks (level >= 1) are addressed under those objects and have no identifier of
  their own.

------------------------------------------------------------
```

2.3 Object taxonomy (canonical vocabulary)

```
These are the canonical nouns for agent state, and the only terms this document
uses for it. Two adjacent categories are deliberately NOT in this list and are
not agent-state object classes: (a) STRUCTURAL plane objects: the logical rootfs
tree encoding (§3), the physical layout encoding (§4), and the runtime-state blob
and memory layout (§5), which are Terrapin objects, each defined in its own
section; and (b) COMMITTED FIELDS: sandbox pin, workload identity, run mode, and
producer, which are committed inside a descriptor or a State Root, not stored as
standalone objects. Terrapin proof (hash-file) blocks are NOT objects: they are
```

```
the level >= 1 blocks of an owning object, addressed by BlockRef and verified
through that object's manifest/tree (§2.2 TERRAPIN-6), never independently
identified. Local cache keys (§9) are derived, not objects. Synonyms used in earlier drafts (chunk, chunk map, artifact,
↪  plane
manifest, opaque checkpoint, golden memory as a distinct object) are retired; see
the Glossary (§17) for the mapping.

block
  A 2,097,152-byte Terrapin block (the final block of a level MAY be short): data
  at level 0, hash-file/proof at level 1+. The unit of fetch and verification into
  the node CAS. A block is addressed within an object as object-digest / level /
  blockIndex (a BlockRef); it has no independent identifier (TERRAPIN-6).

content object
  A Terrapin object whose bytes are exactly the content of one logical regular
  file. Its identifier is terrapin-sha256 over the file's logical (hole-expanded)
  content bytes, independent of path, mode, owner, mtime, xattrs, or which image
  references it. The unit of filesystem dedup. Whole-file: sub-file
  content-defined chunking is NOT used (it is a reserved future layout). Two
  files with identical content resolve to one content object within a cache
  domain.

base rootfs
  The read-only, deduped filesystem base. It is (a) a canonical logical rootfs
  tree: paths, types, modes, owners, mtimes, xattrs, symlink/device/FIFO nodes,
  hardlink groups, encoded as LLT1, yielding the logicalRootfsDigest (§3); and
  (b) that tree's regular-file content realized as content objects via the
  content-addressed physical layout LLAY1, yielding the layoutDigest (§4). Shared
  across agents within a cache domain.

base memory snapshot
  The read-only guest-memory image of a warmed sandbox (e.g. an agent runtime
  that has loaded its interpreter, framework, and client and reached a restore
  point), stored as a single Terrapin object in guest-address-linear order
  (offset 0 in the object is guest offset 0). Identity is terrapin-sha256 over
  the canonical image bytes. Shared copy-on-write across agents (§5). "Golden"
  is an informal adjective for a base memory snapshot of a pre-warmed shared
  runtime; it is not a distinct object class.

base descriptor
  The signed identity object binding one base, as one unit: its OCI image digest,
  base rootfs (logicalRootfsDigest + layoutDigest), base memory snapshot, the
  non-memory runtime-state blob needed to resume (threads, fds, namespaces,
  vCPU/sentry state), the memory layout, the sandbox pin (§2.5), the runtime
  compatibility matrix (§11.4), the assurance mode, and the runtime-state policy
  (the §15.5 ENC-BD-5 no-runtime-state assertion). (The full committed field
  list and encoding are §2.5 DIGEST-BIND-3 and §15.5 LLBD1.) A base descriptor is builder- or derived-attested and is the
  shared, multi-agent base identity. The (rootfs + memory + runtime-state +
  layout) tuple MUST NOT be recombinable across bases (it is committed as a
  unit).

overlay
  A live, per-agent writable layer over a base while the agent runs: copy-on-
  write dirty guest-memory pages plus the writable filesystem upper. An overlay
  is ephemeral (RAM and node-local scratch); it MUST NOT be written to the shared
  CAS and receives no dedup or warmth credit while the agent is running.

delta
  The persisted, content-addressed snapshot of an overlay, captured at hibernate
  or fork. It has three parts. The MEMORY DELTA is a page MAP: for each diverged
  guest page, the source of its bytes (this delta's data object, an ancestor's, the
  base, or the canonical zero) and range, so every page resolves by direct
  reference with no ancestor traversal (§6.3) and a fork shares parent/base pages
  zero-copy (§15 LLMD1). The FILESYSTEM DELTA is a canonical overlay-delta object
  that reconstructs the writable upper completely: content-object references for
  changed regular-file data, plus every overlay namespace and metadata operation:
  created/changed directories, symlinks, device and FIFO nodes,
  ownership/mode/mtime/xattr changes, renames, and deletions (whiteouts/
  tombstones). (Copy-up breaks hardlink identity, as in default OverlayFS; the
  delta records files by content, not hardlink groups: §15 LLFD1.) The RUNTIME DELTA
  is an opaque runtime-native object capturing the diverged non-memory runtime state
  required for exact process resume (DELTA-2R, §15 LLRD1). A delta is per-agent and
  private: it lives in per-cache-domain-keyed CAS and is not deduped across cache
```

```
  domains.

State Root
  The content-addressed identity of one persisted agent snapshot. It is a
  Terrapin object whose canonical encoding (§15) binds: the actor identity; a base
  descriptor reference; a memory delta reference; a filesystem delta reference; a
  runtime delta reference; the sandbox pin; the workload identity; the run mode; the
  producer; the production time; and a parent State Root reference (the lineage edge). A State Root is minted at suspend,
   ↪  hibernate, or
  fork, never for a merely-running agent (whose live per-agent state is an
  ephemeral overlay). It is the unit of resume identity and of forking.

lineage
  The parent-pointer DAG over State Roots. Because a State Root's encoding
  commits its parent, a State Root identifier transitively commits to its
  ancestry. A fork creates a child State Root whose parent is the source snapshot
  and whose memory, filesystem, and runtime deltas begin as copy-on-write references
  to the source State Root's delta components (§6).

Relationships (informative):

  base descriptor (shared, builder-attested)
       ▲
       │ a State Root references its base descriptor
       │
  State Root (per-agent, runtime-attested) ──► parent State Root ──► … (lineage)
       │
       ├──► memory delta      (page-indexed)          per-agent, private
       ├──► filesystem delta  (overlay-delta object)  per-agent, private
       └──► runtime delta     (opaque runtime-native) per-agent, private

  running agent   = base (shared, copy-on-write) + overlay (ephemeral, not stored)
  persisted agent = State Root = base descriptor ref + delta (memory + filesystem +
                                 runtime) + lineage

OBJ-1:
  Every stored LLIFS object, an agent-state object (above) or a structural plane
  object (§3, §4, §5), MUST be a Terrapin object identified per §2.2. Hash-file/
  proof blocks are not objects (they are the level >= 1 blocks of an owning object,
  addressed by BlockRef, §2.2 TERRAPIN-6); committed fields (§2.3 intro) are not
  separate objects.
OBJ-2:
  A content object's identity MUST be over hole-expanded logical content bytes
  only; path, metadata, and hole structure MUST NOT affect it (§4).
OBJ-3:
  A base memory snapshot MUST be stored in guest-address-linear order; pages MUST
  NOT be permuted in the stored object (addresses are load-bearing). Restore
  ordering is a fetch schedule, never a permutation of the object (§5).
OBJ-4:
  A running agent's live per-agent state MUST be an overlay (ephemeral, not in the
  shared CAS). Persisted per-agent state MUST be a delta referenced by a State
  Root.
OBJ-5:
  A State Root MUST commit actor, base descriptor, memory delta, filesystem delta,
  runtime delta, sandbox pin, workload identity, run mode, producer, created, and
  parent; absent components MUST be the explicit "none" value, never omitted (§15).
OBJ-6:
  Identical objects (content objects, memory-delta, filesystem-delta, and
  runtime-delta
  components, base objects) MUST dedupe to one CAS entry per cache domain (§9).

------------------------------------------------------------
```

2.4 Two granularities (the density principle)

```
LLIFS deliberately decouples two units that earlier designs conflated:

  block (2 MiB): the unit of fetch and verification into the node CAS.
  page  (4 KiB): the platform MMU page; the unit of per-agent copy-on-write
                 sharing for the memory plane.
```

Dedup is achieved structurally, not by hashing ever-finer pieces:

- filesystem plane: by the content object (whole-file identity, §4);
- memory plane: by copy-on-write page sharing of one verified base across agents (§5), not by content-addressing each agent's pages.

This is why the memory base uses the ordinary 2 MiB profile yet still shares at 4 KiB: once a 2 MiB base block is verified-resident, its pages are shared copy-on-write across sandboxes with no re-fetch and no re-verification.

2.5 Binding chain

Internal Terrapin identities MUST be bound to a trusted root. There are two binding regimes, and they MUST NOT be substituted.

Builder-attested regime (the shared base). The signed BASE DESCRIPTOR is the attested object that ties the whole base back to the OCI trust root; it is the single signature subject:

```
  OCI sha256 image digest
    → deterministic materialization (§3) → logicalRootfsDigest
      → layoutDigest (a physical layout implementing it, §4)
        → content objects
  base memory snapshot + runtime-state blob + memory layout + sandbox pin (§5)
    │
    └── the signed base descriptor commits ALL of the above as one unit (
        OCI digest, logicalRootfsDigest, layoutDigest, base memory snapshot,
        runtime-state blob, memory layout, sandbox pin, compatibility matrix,
        assurance mode, and runtime-state policy) so none can be recombined and the
        base memory snapshot
        (not derivable from the rootfs digests) is bound to the same OCI subject.
```

Runtime-attested regime (per-agent state). A memory delta, filesystem delta, runtime delta, and State Root are produced at runtime; they have no OCI subject and no builder.
Their integrity is unchanged (Terrapin all the way down); their provenance anchor is a runtime ATTESTATION ENVELOPE:

- signer: the producing node OR the control plane, each holding a key issued by a configured trust root within the node trust boundary. The signer MUST equal the producer recorded in the State Root, or be authorized by policy to attest on that producer's behalf;
- subject: the State Root identifier (which, being content-addressed, transitively covers ALL committed State Root fields: actor, parent, base-descriptor reference, all three delta references (memory, filesystem, runtime), sandbox pin, workload identity, run mode, producer, and created);
- validation: a verifier MUST check the envelope signature against a configured runtime trust root before exposing any runtime-produced byte, AND MUST independently validate the referenced base descriptor under the builder-attested regime (a trusted State Root does NOT confer trust on its base; both checks are required).

Sandbox pin: the runtime assets that produced a snapshot (e.g. the runsc binary) are content-addressed in the EXTERNAL plane by sha256 (they are ordinary fetched assets), recorded per architecture. The pin is a committed field of the base descriptor and of any State Root whose memory is restorable, so a restore on any node selects a compatible runtime and the scheduler can constrain placement (§5, §11). Its canonical encoding is defined in §15.

DIGEST-BIND-1:
  A trusted materialization binding MUST bind an OCI sha256 image digest to a logicalRootfsDigest.
DIGEST-BIND-2:
  A physical layout MUST declare the logicalRootfsDigest it implements; a verifier MUST reject a layout whose declared logical digest does not match.
DIGEST-BIND-3:
  The signed base descriptor MUST be the single attested object binding, as one unit, the OCI image digest, logicalRootfsDigest, layoutDigest, base memory

```
  snapshot, runtime-state blob, memory layout, sandbox pin, compatibility matrix
  (§11.4), assurance mode, and runtime-state policy (§15.5 ENC-BD-5).
  These MUST NOT be recombinable across base descriptors, and the base memory
  snapshot MUST be bound to the same OCI subject through this descriptor (it is
  not derivable from the rootfs digests).
DIGEST-BIND-4:
  A State Root, and the deltas it references, are runtime-attested by the
  attestation envelope (§2.5): a verifier MUST validate the envelope against a
  configured runtime trust root before exposing any runtime-produced byte.
  Builder-assurance modes MUST NOT be claimed for runtime-produced state.
DIGEST-BIND-5:
  The builder-attested base and the runtime-attested per-agent state are distinct
  regimes and MUST NOT be substituted for one another.
DIGEST-BIND-6:
  Every byte exposed to a workload MUST be verified against a Terrapin identifier
  reachable from a trusted, validated root: a base descriptor validated under the
  builder regime, or a State Root validated under the runtime regime. A trusted
  State Root does NOT confer trust on its referenced base descriptor; restore MUST
  validate the base descriptor independently (DIGEST-BIND-3) before exposing any
  base byte.
DIGEST-BIND-7:
  A State Root whose memory is restorable, and the base descriptor it references,
  MUST commit a per-architecture sandbox pin; restore MUST reject an incompatible
  runtime and placement MUST honor it.
```

---

3. Materialization and the logical rootfs tree (LLT1)

The base rootfs (§2.3) has two identities: a LOGICAL identity (the filesystem tree, independent of storage) and a PHYSICAL identity (how it is stored as content objects, §4). This section defines the logical identity and the deterministic function that produces it from an OCI image.

3.1 OCI rootfs materialization

The materialization function maps an OCI image to a canonical logical rootfs tree. Identifier: llifs-oci-rootfs-materialization-v1.

Inputs: OCI image manifest digest and bytes; config descriptor and bytes; ordered layer descriptors; compressed layer blobs; uncompressed diff_ids; target platform. Output: the canonical logical rootfs tree (encoded per §3.2) and its logicalRootfsDigest.

```
MAT-1:  The target platform MUST be resolved before materialization.
MAT-2:  Layers MUST be applied in OCI manifest order for the selected platform.
MAT-3:  Each external OCI descriptor MUST be verified in the external plane
        (sha256).
MAT-4:  Each compressed layer MUST decompress to bytes whose digest matches the
        corresponding config diff_id; a mismatch MUST reject materialization.
MAT-5:  Tar entries MUST be interpreted as byte paths; implementations MUST NOT
        perform Unicode normalization.
MAT-6:  Absolute paths, paths containing a NUL byte, and paths containing ANY
        ".." component MUST be rejected. LLIFS does not resolve "..": rather than
        canonicalize "a/../b", any ".." component rejects. This removes
        normalization ambiguity and rejects root escape with it.
MAT-7:  Path normalization MUST, component-wise: (a) drop "." components;
        (b) collapse repeated "/" to a single "/"; (c) strip any trailing "/".
        It MUST NOT change the byte identity of valid components. The result is
        relative to root (no leading "/", no trailing "/", single "/"
        separators), and directory and file entries normalize identically.
MAT-8:  Within and across layers, a later entry for the same normalized path
        replaces an earlier one, subject to whiteout/opaque semantics.
MAT-9:  Whiteouts apply OCI semantics: ".wh.<name>" removes <name> from the
        lower-visible tree; ".wh..wh..opq" makes the containing directory opaque.
MAT-10: Whiteout markers MUST NOT appear as visible entries.
```

```
MAT-11: Opaque-directory markers MUST NOT appear as visible entries.
MAT-12: Regular files, directories, symlinks, hardlinks, device nodes, FIFOs,
        ownership, mode bits, xattrs, mtimes, and link relationships MUST be
        represented when runtime-visible.
MAT-13: Symlink targets MUST be stored as raw link bytes, unresolved.
MAT-14: A hardlink whose target is absent or unsupported MUST reject.
MAT-15: Hardlinks MUST be represented as the same content object plus a shared
        hardlink group, never as duplicated content.
MAT-16: Unsupported tar/PAX/filesystem metadata that affects runtime-visible
        semantics MUST reject, not silently drop.
MAT-17: Sparse files MUST be represented deterministically (§4 SPARSE).
MAT-18: Device-node major/minor MUST be represented deterministically.
MAT-19: logicalRootfsDigest MUST be independent of physical packing, fetch
        ordering, registry compression, and cache state.
MAT-20: Materialization MUST be deterministic: given the same OCI image digest,
        platform, and function version, all conforming implementations MUST
        produce the same logicalRootfsDigest or reject. The byte-exact encoding
        of §3.2 is what makes this portable across implementations.

Assurance modes (how OCI↔rootfs equivalence is established; a committed field of
the base descriptor, §2.5):

  asserted        A signer asserts OCI → logicalRootfsDigest → layoutDigest. The
                  node verifies signatures and Terrapin bytes but does not
                  re-derive the rootfs. Fastest, weakest; MUST be reported as a
                  weaker mode and MUST NOT claim node-verified equivalence.
  derived-attested  (default) An independent verifier recomputes
                  OCI → logicalRootfsDigest and signs an attestation naming the
                  OCI digest, function id, logicalRootfsDigest, verifier identity,
                  time, and toolchain version.
  node-derived    The node re-derives logicalRootfsDigest from OCI layers before
                  first use; strongest, most expensive; SHOULD run off the TTFE
                  critical path. A recomputed digest differing from the declared
                  one MUST reject and emit a security event.

------------------------------------------------------------
```

3.2 Canonical logical rootfs tree encoding (LLT1)

```
logicalRootfsDigest = terrapin-sha256( LLT1-encode(canonical tree) ). JSON or any
other interchange form MUST NOT affect the digest; the LLT1 byte stream is the
only input to the Terrapin computation.

Primitive encodings (shared with LLAY1, §4.3):

  PRIM-1: u8/u16/u32/u64 are unsigned little-endian, fixed width.
  PRIM-2: i64 is signed two's-complement little-endian (used for mtime seconds).
  PRIM-3: varbytes = a length prefix followed by exactly that many raw bytes;
          path, symlink target, and xattr key/value use a u32 length prefix.
  PRIM-4: all multi-byte integers are little-endian regardless of host.
  PRIM-5: a 32-byte digest field is the RAW Terrapin identifier: the 32-byte
          G(manifest) payload itself, never the "terrapin-sha256:<hex>" string and
          never the bare recursive tree root. ("raw Terrapin identifier" / "raw
          TerrapinID" mean exactly this throughout §3 and §4.)

Document framing:

  magic       4 bytes = "LLT1" (0x4C 0x4C 0x54 0x31)
  version     u16 = 1
  entryCount  u64
  entries     entryCount EntryRecords, sorted

  LLT1-DOC-1: entryCount MUST equal the number of EntryRecords, AND the byte
              stream MUST end exactly after the last entry; a count mismatch or
              trailing bytes MUST reject (length framing detects truncation).
  LLT1-DOC-2: entries MUST be sorted by normalized path bytes, bytewise
              lexicographic, directories and files in one global order; the root
              directory (empty path) sorts first.
  LLT1-DOC-3: normalized paths MUST be unique within the tree; two entries with
              the same normalized path MUST reject (materialization resolves
              same-path collisions per MAT-8 before encoding; the encoded LLT1
```

```
              tree is already collision-free).

Entry record, common prefix (every entry, this exact order/width):

  pathLen u32; path (pathLen bytes, normalized); type u8 (1=dir 2=regfile
  3=symlink 4=chardev 5=blockdev 6=fifo); mode u32 (low 12 bits significant);
  uid u32; gid u32; mtimeSec i64; mtimeNsec u32; hardlinkGroup u64 (0 = not
  hardlinked); xattrCount u32; xattrs (xattrCount XattrRecords, sorted by key).

Type-specific tail (immediately after the prefix):

  regfile  : contentLen u64; contentId 32 bytes (raw TerrapinID of the content
             object, §4.1: not hex, not the bare tree root)
  symlink  : targetLen u32; target (raw link bytes, unresolved)
  chardev  : major u32; minor u32
  blockdev : major u32; minor u32
  dir/fifo : (no tail)

  LLT1-ENTRY-1: fields MUST appear in exactly this order and width.
  LLT1-ENTRY-2: type outside 1..6 MUST reject; sockets and any other type are
                unrepresentable and MUST reject.
  LLT1-ENTRY-3: mtimeNsec >= 1_000_000_000 MUST reject.
  LLT1-ENTRY-4: for regfiles, contentLen MUST equal the content object's length
                and contentId MUST be its 32-byte TerrapinID.
  LLT1-ENTRY-5: mode MUST be (st_mode & 0o7777): perms + setuid/setgid/sticky;
                type bits MUST be masked off (type is carried separately).
  LLT1-ENTRY-6: a hardlink materializes as a regfile sharing the target's content
                object AND inode metadata (mode/uid/gid/mtime/xattrs); only path
                and hardlinkGroup distinguish members.

XattrRecord: keyLen u32; key; valLen u32; val.
  LLT1-XATTR-1: XattrRecords MUST be sorted by key bytes, bytewise lexicographic.
  LLT1-XATTR-2: keys MUST be unique within an entry; a duplicate key (including
                duplicate PAX records resolving to the same key) MUST reject.
  LLT1-XATTR-3: an xattr key is derived from a PAX "SCHILY.xattr.<name>" record by
                stripping exactly the literal prefix "SCHILY.xattr."; the stored
                key is the remaining bytes <name> verbatim (the full namespaced
                attribute name, e.g. "user.foo", "security.capability"), with no
                Unicode normalization. An empty key MUST reject. No namespace
                filtering is applied at the identity layer (every namespace is
                represented as raw key bytes); namespace policy, if any, is an
                admission concern, not part of logicalRootfsDigest.

Path normalization (exactly MAT-6/MAT-7): raw bytes, no Unicode normalization;
relative to root (no leading "/", no trailing "/", single "/" separators); drop
"." components and collapse "//" without altering valid component bytes; reject
absolute paths, NUL bytes, and ANY ".." component. The root is the single
empty-path type-1 entry (attributes from a "./" tar entry if present, else
defaults mode 0o755, uid 0, gid 0, mtime 0, no xattrs).

Hardlink groups: members share contentId and a non-zero hardlinkGroup; ids are
assigned deterministically in sorted path order starting at 1 (the lowest-path
member of each set gets the next id); non-hardlinked entries use 0; a hardlink to
an absent target MUST reject.

Whiteouts/opaque/sparse: the canonical tree is the RESULT of applying layers;
markers MUST NOT appear as entries; sparse follows §4 (hole structure is derived
canonically from content and does not affect contentId; source-archive sparse
metadata is non-portable and MUST NOT be used for identity).

Tar/PAX corner cases: PAX extended headers override ustar fields when present for
path, linkpath, uid, gid, size, mtime, and SCHILY.xattr.*; GNU/PAX sparse headers
are NOT used for identity; tar type map: '0'/'\0'→regfile, '5'→dir, '2'→symlink,
'1'→hardlink, '3'→chardev, '4'→blockdev, '6'→fifo, '7'→regfile, any other →
reject. The tar parser MUST preserve raw path/linkname bytes including embedded
NUL; truncating at NUL before the NUL rejection (MAT-6) fires is a
path-confusion hazard.

PAX numeric and timestamp canonicalization (these feed logicalRootfsDigest, so
they MUST be exact):

  LLT1-NUM-1: integer PAX values (uid, gid, size) are base-10 ASCII. Parsing MUST
              accept an optional leading '-' only where the field is signed,
```

```
              accept leading zeros, and otherwise require digits [0-9] only; any
              other byte MUST reject. uid/gid MUST fit u32 and size MUST fit u64,
              else reject (overflow MUST NOT wrap or saturate).
  LLT1-NUM-2: uid and gid are unsigned; a negative value MUST reject. size MUST be
              non-negative and MUST equal the content object's logical length.
  LLT1-TIME-1: mtime is a base-10 decimal "seconds[.fraction]". The integer part
              is parsed into mtimeSec as a two's-complement i64 (negative,
              pre-1970, allowed); overflow MUST reject.
  LLT1-TIME-2: the fraction is interpreted at nanosecond resolution by FLOORING:
              mtimeSec = floor(value) and mtimeNsec = round-toward-zero of
              (value - floor(value)) * 1e9, so mtimeNsec is always in
              [0, 1e9). Fractional digits beyond 9 are truncated, not rounded. A
              value with no fraction yields mtimeNsec = 0. (Flooring makes
              negative timestamps unambiguous: "-1.5" → mtimeSec -2,
              mtimeNsec 500000000.)
  LLT1-NUM-3: ustar octal fields are used only when no PAX override is present;
              they follow standard ustar octal parsing and the same range checks.
```

---

4. Filesystem plane: content objects and the physical layout (LLAY1)

The logical rootfs (§3) is realized physically as content objects under the content-addressed layout. This is the read path's source of filesystem bytes and the unit of cross-image dedup.

4.1 Content objects

```
CO-1: Every logical regular file MUST have exactly one content-object identifier
      (§2.3) over its content bytes, and in v1 those bytes are stored as a CAS
      content object under that identifier. (Inlining file bytes into the base
      descriptor is reserved, not used in v1: SMALL-2.)
CO-2: A content object's identifier MUST be terrapin-sha256 over the file's
      logical (hole-expanded) content bytes, independent of image, layer, path,
      mode, owner, mtime, or xattrs.
CO-3: Two files with identical content MUST resolve to the same content-object
      identifier; within a cache domain they MUST resolve to the same CAS entry,
      proof blocks, and warmth credit.
CO-4: Hardlinked files MUST reference the same content object; hardlink group
      identity is a property of the logical tree (§3), not the content object.
CO-5: A content object is blocked from offset 0 at the fixed 2 MiB size. A file
      smaller than one block is a single short block, so a cold read of a small
      file fetches only that file's bytes. A zero-length file has no data blocks
      and resolves to the empty content object, whose Terrapin identifier is the
      canonical manifest digest for length 0 (with tree root G(empty)); the bare
      tree root is never the identifier (§2.2).
CO-6: Content-object construction MUST be deterministic.
CO-7: Sub-file content-defined chunking MUST NOT be used; it is a reserved future
      layout and MUST NOT change the content-object identity defined here.

Sparse files and zero blocks:

SPARSE-1: The content-object identifier MUST be over hole-expanded logical bytes; hole
          structure MUST NOT affect the content-object identifier.
SPARSE-2: Hole structure MUST be derived canonically from content (a
          2 MiB-block-aligned all-zero region is canonically a hole), not from
          the source archive's sparse encoding.
SPARSE-3: The canonical zero block is an all-zero 2 MiB Terrapin block; its leaf
          G is the constant in §2.2/§15. This constant is a VERIFICATION value,
          not a stored CAS object: fully-zero block-aligned ranges are never
          stored (SPARSE-4) and are represented as virtual-zero ranges (SPARSE-7).
          A node holds at most the single constant, never per-occurrence copies.
SPARSE-4: Physical storage MUST NOT materialize hole/zero blocks; the layout
          references the canonical zero block for fully-zero ranges.
SPARSE-5: A zero-hashing shortcut MUST produce the identical content-object
          identifier that full hole-expanded hashing would (preserves MAT-20).
```

```
SPARSE-6: Runtime SEEK_HOLE/SEEK_DATA follows the canonical hole structure.
SPARSE-7: A canonical zero range is a virtual verified-zero range derived from
          (zeroLeafDigest, length, cache domain); it requires no stored bytes and
          creates no ordinary object-root CAS identity, lease, or warmth.
SPARSE-8: Virtual zero applies to PROOF blocks too, not just data. A hash-file/proof
          block (level >= 1) whose entries are fully determined by canonical zero
          subtrees MAY be represented virtually: it MUST NOT be stored or fetched, and
          MUST verify byte-identically to the materialized Terrapin hash-file block it
          stands for. This is required for very large sparse objects (notably the
          guest-address-linear memory image, MEM-1, where a high-address mapped-data
          block's proof siblings may be entirely zero subtrees): a verifier resolves
          such proof blocks from the zero constant (§2.2), never the network. The
          all-zero subtree root at each level MUST be derived by the Terrapin
          node-hash recurrence over identical zero-child digests: the level-0 zero
          root is the §2.2 zero-leaf constant, and a level-L zero root is the Terrapin
          node hash over the profile's fanout of level-(L-1) zero roots. §15.10
          ENC-CONF-2 MUST pin the zero-subtree root for each level the profile uses,
          so independent implementations cannot invent divergent shortcut rules; a
          conformance vector MUST exercise a high-address mapped-data region behind a
          large zero span.

Small files:

SMALL-1: Every regular file MUST be a content object regardless of size (a tiny file
         is one short block, CO-5), maximizing cross-image dedup of shared small
         files. Startup latency from many small files is addressed in the transport
         plane by packs (§7; startup cohesion is reserved in v1, §4.4), NOT by
         changing file identity.
SMALL-2: Inlining file bytes into the base descriptor is RESERVED and MUST NOT be
         used in v1. Rationale: it defeats dedup for the highest-fan-out small files
         (an inlined stream earns no CAS presence or warmth, working against the
         density thesis), bloats the hot, must-verify-on-every-restore base
         descriptor, and couples profiling to base identity (the separation §4.4
         COH-3 and LAYOUT-5 establish). If ever reintroduced, inline bytes MUST be a
         bootstrap-bundled Terrapin object that preserves content-object identity and
         CAS promotion, never raw bytes committed into LLBD1.
SMALL-3: (Reserved with SMALL-2.) Were inline reintroduced, an inlined file would
         still be identified by its content-object identifier and verified
         (terrapin-sha256(bytes) == the committed contentDigest) before exposure
         (DIGEST-BIND-6).
SMALL-4: Grouping multiple distinct files into one object MUST NOT be used as a dedup
         mechanism (group identity depends on all members). A startup-cohesion object
         (reserved in v1, §4.4) would likewise be a transport optimization with no
         dedup or identity role, not such a grouping.
SMALL-5: (Reserved with SMALL-2.)

------------------------------------------------------------
```

4.2 Physical layout function (content-addressed)

```
Identifier: llifs-layout-content-addressed-v1. Regular-file data is stored as
content objects; the tree metadata (paths, modes, owners, mtimes, xattrs,
symlink/device/FIFO nodes, hardlink groups) remains base-descriptor metadata
(§3), not deduped as file data; startup ordering is a fetch schedule (§7), not a
physical repack.

LAYOUT-1: A layout MUST declare the logicalRootfsDigest it implements.
LAYOUT-2: A no-hole regular file MUST be a single identity extent with
          objectOffset == fileOffset == 0 and object == the file's content object.
LAYOUT-3: Sparse files MUST be represented per SPARSE (virtual-zero extents,
          canonical zero block).
LAYOUT-4: layoutDigest MUST commit the full file → content-object → extent
          mapping.
LAYOUT-5: layoutDigest MUST NOT commit fetch ordering, pack membership, cache
          state, or peer availability; two deployments with different startup
          schedules over the same files MUST produce the same layoutDigest.

------------------------------------------------------------
```

4.3 Canonical physical layout encoding (LLAY1)

```
layoutDigest = terrapin-sha256( LLAY1-encode(layout) ), reusing the §3.2
primitives. All 32-byte digest fields are raw Terrapin identifiers (G(manifest)),
never hex and never the bare tree root.

Document framing:

  magic "LLA1" (0x4C 0x4C 0x41 0x31); version u16 = 1;
  layoutFn (varbytes, e.g. "llifs-layout-content-addressed-v1");
  implementsLogical 32 bytes (raw TerrapinID of the logicalRootfsDigest);
  objectCount u64 + ObjectDescriptors (sorted by digest bytes, unique);
  fileCount u64 + FileLayouts (sorted by normalized path bytes);
  inlineCount u64 + InlineRecords (sorted by path).

  (Startup-cohesion objects, reserved in v1 (§4.4), are in any case NOT part of
  LLAY1 / layoutDigest: were they defined they would be a profiler-driven transport
  optimization committed by the startup profile/pack (§7), never changing
  layoutDigest, per LAYOUT-5.)

  ObjectDescriptor: digest 32; blockSize u64 (2097152); length u64; tree 32;
                    class u8 (1=content object; other values reserved, MUST reject
                    in this layout).
  FileLayout: pathLen u32; path; size u64; backing u8; then EXACTLY the tail for
              that backing value and nothing else (no sentinel/placeholder for the
              non-applicable index):
                backing==1 (object): objectIndex u64; extentCount u32; extents.
                backing==2 (inline, RESERVED in v1, LLAY1-11): inlineIndex u64;
                                  (extentCount u32 = 0; no extents).
                backing==3 (empty):  (extentCount u32 = 0; no extents; no index).
  Extent: kind u8 (1=data 2=virtual-zero); objectOffset u64; fileOffset u64;
          length u64.
  InlineRecord: pathLen u32; path; contentDigest 32; size u64.

  LLAY1-1: all counts MUST equal the records that follow, AND the byte stream MUST
           end exactly after the last record; trailing bytes MUST reject.
  LLAY1-2: implementsLogical MUST equal the LLT1 logicalRootfsDigest; a verifier
           MUST reject on mismatch.
  LLAY1-3: ObjectDescriptor.digest MUST equal G(manifest(blockSize,length,tree));
           blockSize MUST be 2097152; the bare tree root MUST NOT appear as an
           identifier. ObjectDescriptors MUST be sorted by digest bytes and
           unique by digest.
  LLAY1-4: for backing==1, extents MUST tile [0,size) exactly (sorted by
           fileOffset, contiguous, non-overlapping, no gaps, no coverage past
           size); for content-addressed layout objectOffset MUST equal fileOffset.
           For backing==2 and backing==3, extentCount MUST be 0.
  LLAY1-5: a virtual-zero extent (kind==2) MUST align to the object's zero-block
           leaves, consume no stored bytes, and set objectOffset == fileOffset
           (the same identity-mapping rule as data extents; there is no separate
           object backing for zero ranges).
  LLAY1-6: backing==3 (empty file) has size 0 and no extents; hardlinked files
           share one objectIndex.
  LLAY1-7: layoutDigest MUST commit layoutFn, implementsLogical, all object
           descriptors, all file layouts/extents, and all inline records, and
           nothing about fetch order, pack/cohesion membership, cache state, or
           peer availability (LAYOUT-5).
  LLAY1-8: unknown values of backing, Extent.kind, or ObjectDescriptor.class MUST
           reject. FileLayout paths and InlineRecord paths MUST each be unique and
           sorted by normalized path bytes; an inline path MUST NOT duplicate an
           object-backed file path.
  LLAY1-9: every index MUST reference an existing record of the exact required
           class: FileLayout.objectIndex → an ObjectDescriptor with class==1;
           FileLayout.inlineIndex → an InlineRecord. An out-of-range or
           wrong-class index MUST reject.
  LLAY1-10: every ObjectDescriptor MUST be referenced by at least one
           FileLayout.objectIndex, and every InlineRecord by at least one
           FileLayout.inlineIndex; unreferenced descriptors or inline records MUST
           reject (no unreachable records may perturb layoutDigest).
  LLAY1-11: Inline backing is RESERVED in v1 (SMALL-2): inlineCount MUST be 0, no
           InlineRecords may be present, and backing==2 MUST reject. The inlineCount
           field, the InlineRecord layout, and backing==2 are retained for
           forward-compatibility only.
```

4.4 Startup cohesion (reserved in v1)

A startup-cohesion object would concatenate startup-critical small-file bytes into fewer, larger blocks to cut per-block startup overhead. It is RESERVED and MUST NOT be emitted in v1: serving file bytes from a concatenated object requires a committed segment map (content-object id, file offset, cohesion object id, cohesion offset, length) and per-segment rules proving each served segment against its original content-object identity, which v1 does not define. The v1 mechanism for startup small-file fragmentation is the PACK (§7): a pack batches the startup working set of canonical content-object blocks into one fetch, with no alternate byte carrier and no new identity.

COH-1: Startup cohesion MUST NOT be emitted in v1. If introduced later it MUST be a named structural object (LLCOH1) with a canonical segment map and explicit rules that every served segment verifies against its original content-object identity before exposure (DIGEST-BIND-6).
COH-2: A cohesion object, if ever defined, MUST NOT receive dedup or cross-image warmth credit and MUST be additive only: the same content MUST remain addressable by its content-object identifier, never sole-sourced from it.
COH-3: A cohesion object, if ever defined, MUST NOT be part of physical layout identity (MUST NOT appear in LLAY1 / layoutDigest, §4.3) and MUST be committed by the startup profile/pack (§7), so it never changes layoutDigest (LAYOUT-5).

4.5 Mount model and copy-up

imagefsd serves a read-only lower filesystem; the writable upper is OverlayFS, outside the read-only base. The default mount is FUSE; for gVisor the lower is served through a CAS-backed lisafs gofer with no host-kernel FUSE on the path (§11).

MOUNT-1: Mount MUST NOT require full hydration unless materialization strictness requires it.
MOUNT-2: Directory traversal and stat for startup-critical paths MUST be served from local bootstrap metadata before STARTUP_READY.
MOUNT-3: The writable upper is outside the read-only base. While the agent runs the upper is an ordinary OverlayFS upper and is not verified by LLIFS. On suspend, the upper's diff MUST be captured into the filesystem delta (§6) as a canonical overlay-delta object; once captured it is read-only and Terrapin-verified like any other object.

Copy-up is a latency hazard: a one-byte write to a large lower file can force a large synchronous read. Trusted runtime requirements (not untrusted profiles) MAY mark expected-mutable files with a strategy:

prehydrate-full-file	fetch+verify the whole file before STARTUP_READY
seed-upper	synthesize the file into the upper before start
volume-required	redirect the path to writable storage (admission may reject otherwise)
deny	fail fast on mutation
allow-lazy-below-threshold	permit lazy copy-up only below a size

COPYUP-1: Lowerdir mutation MUST be treated as a latency hazard.
COPYUP-2: Mandatory copy-up behavior MUST come from trusted runtime requirements, not from an untrusted profile.
COPYUP-3: A prehydrate-full-file path MUST be fetched and verified before STARTUP_READY; a volume-required large path SHOULD cause admission to reject unless redirected to writable storage.
COPYUP-4: Copy-up full-file fetches MUST be measured separately from ordinary foreground reads.

---

5. Memory plane: base snapshot, copy-on-write restore, restore hazards

The filesystem plane (§§3–4) delivers verified, deduped, lazily-faulted files. The memory plane delivers the same for guest memory: a verified, deduped, lazily-faulted base memory snapshot from which sandboxes restore, sharing the base copy-on-write so a million agents are one shared base plus a small per-agent overlay. This section defines the shared base and the restore protocol; the persisted memory delta and the lifecycle that mints it are §6.

5.1 Base memory snapshot

A base memory snapshot is captured once from a warmed sandbox (interpreter, framework, and client loaded; paused at a restore point) and committed by a base descriptor (§2.3, §2.5). The base descriptor commits: the base memory snapshot OBJECT (the guest memory image, a Terrapin object, §5.3); the runtime-state blob (non-memory sandbox state: threads, fds, namespaces, vCPU/sentry state, needed to resume); and the memory layout (guest-address → object-offset mapping, §5.3). A restore working set (§5.5) is associated PROFILE metadata keyed to the base version, not a committed base-descriptor field.

MEM-1: The base memory snapshot object identifier MUST be terrapin-sha256 over the canonical HOLE-EXPANDED, guest-address-linear byte image of the committed guest address space [0, committedEnd), committedEnd being the end of the highest region in the memory layout (§15.5 ENC-ML-1). Byte offset 0 in this image is guest offset 0 (MLAYOUT-1). Mapped-data ranges contribute their captured bytes; mapped-zero and unmapped/no-access ranges contribute canonical zeros. Identity is over LOGICAL bytes, computed with the canonical zero-block shortcut (the §4 SPARSE rule on the memory plane), so it is independent of physical sparseness and never materializes zero ranges.
MEM-2: The base is captured once per base version, hashed, and signed. Producers are NOT required to reproducibly regenerate an identical image; identity is "snapshot once, content-address the result." (Contrast MAT-20: the filesystem base is reproducibly derivable; the memory base is captured.)
MEM-3: A base memory snapshot is executable state and MUST NOT be restored unless reachable from a trusted, signed base descriptor (DIGEST-BIND-3/6).
MEM-4: In addition to the full base-descriptor binding of DIGEST-BIND-3, the runtime-state blob and memory layout MUST be committed by the base descriptor so the (memory + state + layout) triple cannot be recombined across bases.
MEM-5: The MEM-1 identity commits LOGICAL bytes and does NOT encode mapping state: a mapped-zero range and an unmapped/no-access range hash identically (both zeros). Per-range mapping STATE (§5.3 MLAYOUT-5) is committed ONLY by the signed memory layout, which the base descriptor binds (MEM-4, DIGEST-BIND-3). Restore MUST derive every range's state from the signed layout and MUST NOT infer state from snapshot bytes; two snapshots with identical base_memory but different layout STATE are different bases.

5.2 Storage granularity (the block/page decoupling)

This restates §2.4 normatively for the memory plane.

GRAN-1: The base memory snapshot object MUST use the standard 2 MiB profile; the 2 MiB block is the unit of fetch and verification into the node CAS only.
GRAN-2: The unit of per-agent copy-on-write sharing MUST be the platform MMU page (4 KiB on x86-64; the native page size otherwise). CoW granularity is independent of block size.
GRAN-3: Once a 2 MiB base block is verified-resident in the node CAS, its pages MUST be shareable copy-on-write across sandboxes without re-fetch and without re-verification.
GRAN-4: Implementations MUST NOT introduce a sub-2-MiB memory profile unless measured cold-node over-fetch (§5.5) justifies it; if justified, a finer profile MAY be defined separately and MUST be reported in telemetry. Page

dedup MUST NOT be the justification; dedup is achieved by base sharing
(§5.4), not by content-addressing each agent's pages.

5.3 Memory layout (guest-address-linear)

Memory addresses are load-bearing (pointers); pages MUST NOT be permuted in the
stored object the way file bytes may be repacked (§4). The base memory snapshot
object is stored in guest-address-linear order, which is both canonical for dedup
and trivially MAP_PRIVATE-mappable as one linear region.

MLAYOUT-1: Guest-address-linear: in the canonical hole-expanded image (MEM-1) byte
offset O is guest offset O across the whole committed space
[0, committedEnd), not merely within one region. Physical storage MAY be
sparse (holes for zero, unmapped, and not-yet-fetched ranges); logical
offsets are unaffected.
MLAYOUT-2: A base memory snapshot is exactly ONE Terrapin object (§2.3); multiple
objects MUST NOT be used. Its IDENTITY is the hole-expanded logical byte
image of MEM-1; its PHYSICAL storage is sparse. The memory layout
describes the guest regions and per-range state over that one object.
Mapped-data ranges occupy stored object bytes (resident only after fetch
and verify). Mapped-zero and unmapped/no-access ranges are logical holes:
they contribute canonical zeros to identity (MEM-1) but occupy NO stored
or transferred bytes, and they keep their guest offsets (NOT compacted
out) so the image remains one linear MAP_PRIVATE region (MLAYOUT-1). The
layout MUST be deterministic.
MLAYOUT-3: Restore ordering MUST be expressed only as a fetch schedule (§5.5),
never as a permutation of the stored object (MLAYOUT-1).
MLAYOUT-4: The memory layout MUST commit the guest-region → object-extent mapping
and the page size assumed at capture.
MLAYOUT-5: The memory layout MUST commit, per guest address range, its exact
SIGNED mapping STATE (one of three), and restore MUST reproduce it.
Cache residency (resident vs not-yet-fetched) is an ORTHOGONAL runtime
condition, NOT a signed state:
- mapped-data: backed by base object bytes. When not yet resident, a
touch traps, fetches, and verifies before exposure (COW-6); when
resident, pages are shared copy-on-write (COW-1). "Absent" is this
state's not-yet-resident runtime condition, not a fourth state.
- mapped-zero: the canonical verified zero block (§4 SPARSE-3); a
touch installs a verified zero page (COW-6b), no fetch.
- unmapped / no-access: restore as UNMAPPED/protected. It MUST NOT
become mapped-zero and MUST NOT be lazily fetched; a touch MUST
fault to the guest exactly as it would natively, never silently
resolving to a page.
The layout MUST also commit page protections and mapping attributes
(R/W/X, guard pages, shared vs private), and restore MUST reestablish
them.

5.4 Copy-on-write restore protocol

The node CAS materializes the base memory snapshot object as a sparse memfd/file:
verified 2 MiB blocks are written at their layout offsets; not-yet-fetched blocks
are holes. Each restored sandbox maps the base region MAP_PRIVATE over that file;
the kernel shares resident base pages across all sandboxes and copy-on-writes a
private page on first write. Per-agent dirty pages are the overlay (§6).

Sharing one verified base across sandboxes via MAP_PRIVATE requires the runtime's
memory backing to support a shared, verified base file; this is the one net-new
runtime capability and is specified technically in §11.

CRITICAL: for memory, zero is valid data. A hole meaning "block absent (not yet
fetched)" MUST NEVER be confused with a page that legitimately reads zero. An
unprotected hole resolves to a zero-filled page on touch, silently exposing
unverified, incorrect memory. Every absent base range MUST be trapped (COW-6)
before any sandbox can touch it.

```
COW-1: Restore MUST back the guest base region with a MAP_PRIVATE mapping of the
       verified base file, so resident base pages are physically shared and writes
       fault to private copies.
COW-2: Per-agent dirty pages are the memory overlay (§6); they MUST NOT be written
       to the shared CAS and receive no dedup or warmth credit (they are ephemeral
       until persisted as a memory delta, §6).
COW-3: A node MUST hold at most one physical copy of a given base block's resident
       pages per cache domain, shared across all sandboxes restoring that base.
COW-4: Restore MUST NOT require loading the full base image; only the restore
       working set (eager) plus lazily-faulted pages (COW-6) are required to
       resume (the memory analog of MOUNT-1).
COW-5: Implementations SHOULD cap the per-agent dirty-page set and report agents
       whose overlay growth defeats density (so they can be migrated or
       rescheduled).

Verification:

MVERIFY-1: Every base memory byte MUST be Terrapin-verified at CAS admission,
           before it is ever mapped into any sandbox.
MVERIFY-2: Verification is per-block-per-node and amortized: once a base block is
           verified-resident, per-agent CoW sharing of its pages MUST NOT
           re-verify (GRAN-3). Verification cost does not scale with agent count
           or fault count.
MVERIFY-3: Per-agent dirty pages are agent-generated and MUST NOT be verified
           against any Terrapin identifier; they are not CAS content.

Fault handling:

COW-6:  Absent base ranges MUST be protected so they cannot resolve to zero-filled
        or stale pages. First touch MUST synchronously fault through the LLIFS
        handler, fetch and verify the containing block at P0, ADMIT the verified
        block into the node CAS / shared base backing, then map or continue the
        faulting page from that verified shared backing (preserving the shared-base
        invariant COW-1/COW-3, never a per-sandbox unverified copy), and only then
        resume the faulting thread. A read of an absent range MUST block, never
        return zeros.
COW-6a: Absent (not-yet-fetched) ranges require userfaultfd MISSING-fault handling
        (populate verified content before resuming), NOT minor-fault handling.
        MISSING handling MAY use UFFDIO_COPY only where it populates, or continues
        from, the shared verified base backing (COW-1/COW-3); it MUST NOT install
        an unverified or per-sandbox-private copy that bypasses the shared base.
        Minor-fault / UFFDIO_CONTINUE applies only to ranges already present and
        verified in the shared base backing that are being mapped into an
        additional sandbox. Implementations MUST choose the fault mode by backing
        state, not assume one.
COW-6b: A base block the signed memory layout designates as the canonical zero
        block (§4 SPARSE-3) is verified content, NOT absent: the handler MAY
        install a verified zero page immediately (e.g. UFFDIO_ZEROPAGE) without a
        fetch. "Known-zero per signed layout" MUST be distinguished from "absent"
        (COW-6) and MUST be driven by the signed layout, never inferred from a
        memfd hole.
COW-7:  A hard failure to fetch or verify a faulted base block MUST surface as a
        distinct, process-fatal-class event (cf. §8), not a silent hang, and MUST
        follow the §8 fetch-fallback and hedging rules.
```

5.5 Restore working set (prehydration)

```
Cold-node restore cost is dominated by fetching the base blocks the guest touches
during resume. A restore working set lists those blocks, prioritized, to
prehydrate before resume; the cold tail faults lazily (§5.4). This is the
memory-plane analog of the startup pack; the full profile/pack/coverage machinery
is §7.

MRESTORE-1: A restore working set MUST reference base memory snapshot blocks as
            canonical BlockRefs (object / level / blockIndex).
MRESTORE-2: imagefsd SHOULD prehydrate the working-set blocks into the node CAS
            before resume, within configured byte and latency budgets.
MRESTORE-3: Blocks not in the working set MUST be faultable lazily during
            execution (COW-6) at foreground (P0) priority.
```

```
MRESTORE-4: A restore working set MUST be associated with a base version and a
            memory profile, not only a base object digest.
MRESTORE-5: imagefsd MUST report cold-node memory over-fetch (bytes fetched but
            not touched in the measured restore window); this is the signal that
            decides whether a finer memory profile (GRAN-4) is warranted.
```

5.6 Restore hazards

```
Restoring N agents from one base clones state that MUST NOT be shared. These
hooks are mandatory, not optional.

RHAZARD-1: Entropy MUST be refreshed on restore (kernel RNG, userspace PRNG
           seeds, cached nonces/keys derived at capture). Reusing base entropy
           across agents is a security defect.
RHAZARD-2: Time sources MUST be refreshed (monotonic and wall clocks jump
           relative to the base).
RHAZARD-3: External state captured in the base (open sockets, host fds, in-flight
           connections, leases) MUST be treated as dead and reconnected or
           invalidated.
RHAZARD-4: The base capture point SHOULD precede any agent-, tenant-, or
           request-specific secret entering memory, so the shared base holds no
           cross-agent-sensitive data. For multi-tenant or cross-request reuse
           this strengthens to MUST, or capture-time secret scanning/attestation
           is required.
RHAZARD-5: Restore-hazard handling MUST be part of signed base/runtime policy; a
           base lacking required hooks MUST NOT be used for multi-tenant restore.
RHAZARD-6: Cross-tenant base sharing MUST obey cache domains (§9); a memory base
           is a content-presence and behavioral side channel and MUST NOT be
           shared across domains unless explicitly opted in.
RHAZARD-7: Before HAZARDS_CLEARED for multi-tenant restore, each of the following
           MUST be refreshed/invalidated or asserted not-applicable by signed
           policy: PID/TID and cached process identity; futexes, robust lists,
           rseq, TLS; vDSO/vvar time mappings; timers, epoll/poll, pending
           signals, pending async I/O; ASLR / address-space assumptions (shared
           base ⇒ shared layout; inject new per-agent randomness post-restore,
           RHAZARD-1); page protections and mapping attributes (R/W/X, guard
           pages, no-access ranges, shared vs private) reestablished per MLAYOUT-5;
           seccomp, namespaces, cgroups, credentials, capabilities;
           language-runtime state (Go scheduler/netpoller, JVM safepoints, Python
           hash seed, OpenSSL/BoringSSL DRBG reseed); capture-time secret scanning
           or a no-secrets-before-capture guarantee (RHAZARD-4).
```

5.7 Memory-restore lifecycle

```
  BASE_UNRESOLVED
    → BASE_RESOLVED              (signed base descriptor verified)
    → BASE_STARTUP_PREHYDRATED   (restore working set resident and verified)
    → RESTORED                   (sandbox mapped CoW; base runtime-state then
                                  runtime delta applied, RDELTA-5)
    → HAZARDS_CLEARED            (RHAZARD-1..7 refreshed)
    → RUNNING

Failure states: BASE_DESCRIPTOR_UNTRUSTED, BASE_MEMORY_BLOCK_UNAVAILABLE,
BASE_VERIFICATION_FAILED, RESTORE_HAZARD_HOOK_MISSING, COW_BACKING_UNAVAILABLE.
```

---

6. Agent state and lifecycle

```
An agent is a shared, read-only BASE (base rootfs §4 + base memory snapshot §5,
bound by a base descriptor §2.5) plus a per-agent OVERLAY across memory and
filesystem. This section defines how the overlay is persisted (the delta), how a
snapshot is identified (the State Root) and branched (lineage and fork), how a
```

snapshot is written, and the RAM-reclamation mechanisms (reset, hibernate, yield)
with their storage tiers. It builds directly on §4 (content objects, the writable
upper) and §5 (base memory snapshot, copy-on-write restore, restore hazards).

6.1 Overlay and delta

While an agent runs, its per-agent state is an OVERLAY (§2.3): copy-on-write dirty
guest-memory pages (§5.4) plus the writable filesystem upper (§4.5). The overlay
is ephemeral; it lives in RAM and node-local scratch, is never written to the
shared CAS, and earns no dedup or warmth (COW-2, MOUNT-3).

At suspend, hibernate, or fork, the overlay is captured into a DELTA (§2.3): a
persisted, content-addressed, per-agent object with three parts (memory,
filesystem, and runtime).

DELTA-1: The MEMORY DELTA MUST be a page MAP: for each diverged guest page, the
source of its bytes (this delta's own data object, an ancestor's, the base,
or the canonical zero) and range. The map directly resolves every page, so
resume needs no ancestor traversal (§6.3 LINEAGE-3) and a fork shares
parent/base pages by direct reference with no copy. Pages identical to the
base remain shared from the base copy-on-write (§5.4) and need not be
listed. Its byte encoding is §15 (LLMD1).
DELTA-2: The FILESYSTEM DELTA MUST be the canonical overlay-delta object (§2.3,
§15): content-object references for changed regular-file data, plus every
overlay namespace and metadata operation: created/changed directories,
symlinks, device and FIFO nodes, ownership/mode/mtime/xattr changes,
renames, and deletions (whiteouts/tombstones). It MUST be sufficient to
reconstruct the writable upper exactly. Hardlink identity in the upper is
NOT preserved across copy-up (as in default OverlayFS): copied-up files are
recorded by content, not as a hardlink group; restoring hardlink identity
is a future extension.
DELTA-2R: The RUNTIME DELTA captures the agent's diverged NON-memory, non-filesystem
runtime state: the runtime-native execution state the base runtime-state blob
(§5.1) no longer describes after the agent has run, and which guest memory
alone does not reconstruct (for gVisor e.g. thread/register state, fd tables
and offsets, epoll/timer/signal/futex/rseq/TLS state, syscall-restart state,
VMA metadata, socket state, and sentry/kernel bookkeeping). A memory delta
plus filesystem delta cannot restore a non-cooperating process hibernated
mid-request or yielded while blocked upstream; the runtime delta is what
makes that exact. For v1 it is an OPAQUE runtime-native Terrapin object, like
the base runtime-state blob; LLIFS defines no internal structure (§15 LLRD1).
Its encoding MAY later be optimized into a delta against the base
runtime-state blob; correctness comes first.
DELTA-3: A delta is per-agent and private: it MUST live in per-cache-domain-keyed
CAS and MUST NOT be deduped across cache domains (§9). Each delta
component is a Terrapin object verified before exposure (DIGEST-BIND-6).
DELTA-4: Capturing the overlay MUST occur under the single freeze barrier (WRITE-1)
so the memory, filesystem, and runtime deltas share one freeze epoch;
restore MUST present a coherent memory↔filesystem↔runtime view (no mmap'd
working-plane page may reflect a different epoch than the memory delta).
DELTA-5: A persisted delta (memory, filesystem, or runtime) MUST be directly
resolvable on
the resume critical path: every object it needs is named by absolute
Terrapin id and read directly, with NO ancestor or lineage traversal
(LINEAGE-3, §6.3 FORK-3). Storage MAY share ancestor objects zero-copy, and
a named ancestor OBJECT (the base via base_ref, or an LLMD1 source==2
ancestor delta-data object) is a DIRECT resolution input read without
traversal. The lineage LINKS proper, the parent State Root and an LLFD1
parent_ref, are provenance, authorization, GC, or compaction inputs ONLY;
the resume-critical resolved form (direct component object ids) MUST be
obtainable without walking lineage (e.g. via the bootstrap, §7 BOOT-2).
RDELTA-1: A State Root that persists the memory plane MUST also commit a runtime
delta, UNLESS its base descriptor carries the signed NO-RUNTIME-STATE
ASSERTION. That assertion has one canonical home: the base descriptor's
runtime_state_policy field (§15.5 ENC-BD-5; value 1 asserts that no
non-memory runtime state is required to resume ANY memory-persisting State
Root on this base, the runtime reconstructing all execution metadata from
committed memory/filesystem state and signed policy, NOT merely that the base
restore point held none), committed by the signed base descriptor and so

validated as part of base-descriptor validation (DIGEST-BIND-3/6).
Restore MUST check it BEFORE accepting a memory-persisting State Root whose runtime is "none"; absent it (runtime_state_policy==0, the default), memory non-none with runtime "none" MUST fail closed (RDELTA-5). reset, golden, clean, and filesystem-only persistence need no runtime delta and no assertion (§6.6).

RDELTA-2: The runtime delta is a Terrapin object over runtime-native bytes (§15 LLRD1), interpretable ONLY by a runtime satisfying the sandbox pin (§2.5) and the compatibility matrix (§11.4); LLIFS defines no portable structure for it in v1.

RDELTA-3: The runtime delta MUST be captured under the SAME freeze barrier as the memory and filesystem deltas (DELTA-4, WRITE-1), sharing one freeze epoch.

RDELTA-4: The runtime delta MUST be directly resolvable on the resume critical path with no lineage traversal (DELTA-5).

RDELTA-5: Restore MUST apply the base runtime-state blob THEN the runtime delta before unfreeze, and MUST fail closed on a mismatch or on a missing required runtime delta (RDELTA-1).

6.2 State Root

A State Root (§2.3) is the content-addressed identity of one persisted agent snapshot. It is minted at suspend, hibernate, or fork, never for a merely-running agent. Its canonical byte encoding and identifier (a Terrapin object) are §15; this section fixes the committed fields and their rules.

A State Root commits:

actor	stable agent identity; survives across all snapshots of one agent. A fork allocates a NEW actor (§6.3).
parent	the State Root this derives from, or "none" for genesis; the lineage edge (§6.3).
base	the base descriptor reference (§2.5): the shared base rootfs + base memory snapshot + runtime-state blob + memory layout this snapshot restores onto.
memory	the memory delta reference (DELTA-1), or "none".
filesystem	the filesystem delta reference (DELTA-2), or "none".
runtime	the runtime delta reference (DELTA-2R, RDELTA-1), or "none"; required when the memory plane is persisted unless signed policy proves none is needed (RDELTA-1).
sandbox pin	the per-architecture sandbox pin (§2.5), required when memory is persisted.
workload	the workload identity (SR-5), required when any per-agent plane (memory or filesystem) is persisted.
run_mode	which planes persist (§6.6).
producer	the producing node/control-plane identity; the runtime-attestation subject (§2.5).
created	production time (descriptive).

SR-1: A State Root MUST commit all fields above; absent components MUST be the explicit "none", never omitted. Its identifier MUST be the Terrapin object identifier of its canonical encoding (§15); there is no separate digest scheme (§2.1).

SR-2: A State Root MUST reference a base descriptor; restore MUST validate the State Root under the runtime regime AND the base descriptor under the builder regime (DIGEST-BIND-4/6); a trusted State Root does not confer trust on its base.

SR-3: memory and filesystem are the per-agent state references (the runtime delta accompanies a persisted memory plane per SR-4/RDELTA-1); setting NEITHER memory nor filesystem is valid (a clean or golden-only snapshot, §6.6). A resume reads whichever references are non-"none".

SR-4: A State Root that persists ANY per-agent plane (memory or filesystem) MUST commit the workload identity (SR-5); a delta is workload-specific and MUST NOT be restored against a different workload. A State Root that persists the memory plane MUST additionally commit the sandbox pin (§2.5, DIGEST-BIND-7) and the runtime delta unless signed policy proves none is required (RDELTA-1).

SR-5: The WORKLOAD IDENTITY is a canonical Terrapin digest (§15 LLWI1) over the RESTORE-VISIBLE fields of the control plane's workload spec (atelet.WorkloadSpec): every field that, if different at restore, could

```
      invalidate or make unsafe the reapplication of a persisted memory/filesystem
      delta. It MUST commit, at pod scope: the pause image, hostname, the host- and
      shared-namespace flags, and the pod security context; and per container, in
      declared order: name, image, command, args, env, working directory, run-as
      user/group, capabilities (added and dropped), security profile (privileged,
      read-only-rootfs, no-new-privileges, seccomp/AppArmor/SELinux), mount and
      volume-device topology (mount path, sub-path, read-only, source name/type),
      exposed devices, and the restore-relevant resource limits (at least the memory
      limit, which constrains the guest address space). Fields intentionally EXCLUDED
      (they do not affect restored state or are regenerated post-restore) are
      enumerated in §15 LLWI1 (ENC-WI-2). On restore the supplied workload MUST match
      the committed workload identity exactly; a mismatch MUST reject, never silently
      restore. Any atelet.WorkloadSpec field that affects restore correctness MUST be
      added to LLWI1 and to this list, never silently ignored.
SR-6: created is committed, so it participates in identity: two snapshots of one
      actor collapse to one State Root identifier ONLY when all committed fields,
      including created, are byte-identical (intended idempotent-retry behavior,
      not a collision). Snapshots that differ only in created are distinct State
      Roots; an implementation wanting retries to collapse across a time boundary
      MUST reuse the original created value.

------------------------------------------------------------
```

6.3 Lineage and forking

```
The parent State Root reference forms a hash-linked DAG over State Roots: a State
Root identifier transitively commits to its ancestry. A FORK creates a new actor that begins from
an existing snapshot's exact state, then diverges: the differentiator for
speculative agent branching.

FORK-1: A fork MUST allocate a NEW actor identity and a new State Root whose parent
        is the source snapshot.
FORK-2: base (the base descriptor) MUST be shared by reference, never copied.
FORK-3: The child's memory, filesystem, and runtime deltas MUST begin as
        copy-on-write references to the source State Root's delta components; only
        diverged pages/files MAY be newly stored. No per-plane copy is required at
        fork time. These references MUST be DIRECT and self-contained: a child delta
        names the exact base/ancestor component objects it depends on, so resume
        resolves them by direct reference and never walks the lineage chain
        (LINEAGE-3). A compaction step MAY flatten a chain into direct references,
        but resume MUST NOT depend on traversal.
FORK-4: A fork MUST NOT require rehydrating the source onto the child's node before
        the child can resume; cold pages/blocks fault on demand (§5.4, §8). A fork
        is a read fan-out of one snapshot to N children and SHOULD reuse the
        fan-out, hedge, and seed machinery of §8/§10.
FORK-6: Self-containment (DELTA-5) moves cost off the resume path onto snapshot and
        compaction: producing a self-contained child MAY require O(parent-dirty-pages
        + parent-upper-metadata) work (a large LLMD1 map naming ancestor pages
        directly; a complete LLFD1 upper). This is an intentional tradeoff. For
        high-fan-out speculative branching, a later Merkle-map/trie representation of
        LLMD1/LLFD1 MAY let forked children structurally share maps while staying
        directly resolvable; v1 accepts the flat cost.
FORK-5: A fork's State Root MUST be runtime-attested by the node/control plane
        performing it (§2.5); the verifiable parent link preserves the chain to the
        source. Fork authorization MUST be a control-plane policy decision (who may
        fork actor X); the authorization hook MUST exist. (A deployment MAY relax
        enforcement; phase guidance is §16.)
LINEAGE-1: parent MUST reference a resolvable, verifiable State Root or be "none".
LINEAGE-2: The substrate MUST be able to enumerate a State Root's lineage for GC
           (§9) and revocation (§14).
LINEAGE-3: Lineage depth MUST NOT be on the resume critical path: a resume reads
           the State Root's direct base/memory/filesystem/runtime references, not the
           ancestor chain (§7).

------------------------------------------------------------
```

6.4 Snapshot write/commit path

```
Agents PRODUCE state; the write path is normative. On suspend, hibernate, or fork:
```

1. Freeze the agent (quiesce the runtime).
2. Checkpoint memory; compute the page-indexed memory delta against the base (and parent, for a fork), writing only changed pages (DELTA-1). Capture the runtime delta (the diverged non-memory runtime state, DELTA-2R) unless signed policy proves none is required (RDELTA-1).
3. Capture the writable upper into the filesystem delta (DELTA-2).
4. Write all new objects to the CAS under temporary names and atomically promote each only after Terrapin verification.
5. Compute the memory, filesystem, and runtime delta object identifiers; assemble and identify the State Root (§15).
6. Obtain the runtime attestation over the State Root (§2.5).
7. Publish the State Root to the control plane; demote node-pinned objects toward the cold tier per the run mode and tier policy (§6.6, §9).

WRITE-1: Steps 1–3 MUST occur under a single freeze barrier so the memory, filesystem, and runtime deltas share one freeze epoch (DELTA-4, RDELTA-3, §5).
WRITE-2: New objects MUST be written under temporary names and atomically promoted only after Terrapin verification.
WRITE-3: A State Root MUST NOT be published until every object it references is durably present in at least one tier the resume path can reach.
WRITE-4: The freeze→publish path MUST be crash-safe: a crash before publish MUST leave the prior State Root valid and resumable; partial objects MUST be GC-able garbage (§9), never referenced.
WRITE-5: Snapshot-write latency MUST be reported separately from resume latency (§13).

6.5 Reclamation mechanisms: reset, hibernate, yield

Farm-to-LLM agents are memory-bound and mostly idle (blocked on LLM calls, between turns, or done). The substrate reclaims their RAM via three mechanisms, all of which are §5 restores differing only in which delta is reapplied:

reset	stateless completion → restore the base with NO delta.
hibernate	stateful pause → restore base + the agent's persisted delta.
yield	reclaim during a blocking upstream (LLM) call → hibernate a blocked agent + an upstream transaction that survives the eviction.

DESIGN TENET (non-cooperative baseline): agents run arbitrary libraries that will not cooperate. Cooperation OPTIMIZES; OBSERVATION guarantees. Correctness MUST hold for fully non-cooperating workloads.

ROBUST-1: Cooperation (completion signals, yield hints, an SDK) is an OPTIMIZATION; correctness and acceptable reclamation MUST hold for non-cooperating workloads, which MUST NEVER be corrupted.
ROBUST-2: The substrate observes guest state because the sandbox kernel owns the guest memory, filesystem, and network stack, and an L7 ingress/egress proxy owns the request connection and the upstream call. Together they let the substrate reclaim non-cooperating agents.
ROBUST-3: A completed successful request/response pair marks a candidate request BOUNDARY and MAY trigger reclamation; it does NOT prove the process is safe to reuse (a 2xx can leave leaked secrets, mutated globals, or background work). Therefore the DEFAULT reclamation on a boundary is restore-fresh (RESET-1); failed/errored pairs MUST restore-fresh.

RESET-1: The default reset is restore-fresh: bind the next request to a freshly restored base instance and discard the completed one. The fresh instance shares resident base pages copy-on-write (§5.4), so it is cheap (no per-agent delta to load), NOT a cold start.
RESET-2: Reset MUST reset EVERY mutable plane: guest memory, the filesystem upper, fd table/offsets, task/thread/timer/signal/futex state, sentry/kernel runtime state, and external resources (RHAZARD), not merely drop the memory overlay.
RESET-3: Reset MUST clear RHAZARD-1..7 (§5.6) before the instance serves, or assert each not-applicable by signed base policy.
RESET-4: In-place rewind (rewinding a running process's full state to base in its existing slot) is a DEFERRED optimization, permitted only under a

```
         machine-checkable quiescence predicate AND a measured win over
         restore-fresh; until then the substrate uses restore-fresh.

HIBER-1: A hibernation snapshot captures only the per-agent delta over the
         immutable base (the base is not copied); wake = §5 restore + delta apply.
HIBER-2: The delta MUST be content-addressed and Terrapin-verified; wake verifies
         it before exposure (DELTA-3).
HIBER-3: Wake MUST clear RHAZARD-1..7 (reseed entropy, refresh time, reconnect or
         invalidate external state).
HIBER-4: Eviction policy (idle timeout, memory pressure, explicit yield) MUST be
         trusted policy, not agent-controlled.
HIBER-5: Wake is on the event's critical path at P0 with a wake-latency budget;
         wake latency MUST be reported (§13).
HIBER-6: Hibernation deltas are leased objects (§9); the base refcount MUST reflect
         every agent (active or hibernated) that will wake to it.

YIELD-1: Yield is hibernate applied to an agent blocked on an upstream call, plus
         an upstream transaction. The proxy MUST take durable ownership of the
         upstream request before the agent is evictable; the pre-ack window MUST
         NOT be evictable and pre-ack eviction attempts MUST be observable.
YIELD-2: Upstream SUBMISSION MUST be single-flight under a durable transaction id
         bound to (agent invocation, upstream target, request-body digest, retry
         epoch), issued and durably logged BEFORE submission begins. The proxy MUST
         NOT, on its own initiative, submit the same transaction id more than once,
         and the id MUST NOT deduplicate distinct invocations.
YIELD-2a: Single-flight submission does NOT by itself guarantee at-most-once
         upstream EXECUTION. The proxy's durable log MUST model each transaction as
         one of not_sent, sent_unknown (submitted, outcome unconfirmed: crash or
         ambiguous network failure), response_held (a definitive response captured),
         or definitive_failure. From sent_unknown the proxy MUST NOT silently
         resubmit a non-idempotent call. At-most-once EXECUTION across a sent_unknown
         ambiguity is achievable ONLY with upstream cooperation: an idempotency key
         the upstream deduplicates on, or a cooperative upstream transaction
         protocol. Absent that, per-upstream policy MUST choose between failing
         definitively (no resubmission) and resubmitting (accepting possible
         duplicate execution); the substrate MUST NOT claim general exactly-once
         effects for arbitrary non-idempotent upstreams.
YIELD-3: The agent↔proxy downstream connection is NOT preserved (RHAZARD-3); on
         wake the restored delta resumes its pending read and the proxy delivers
         the single held response (replay for retrying clients; a defined
         continuation or a defined client-visible error otherwise), under a bounded
         hold timeout.

  State machine:
    RUNNING
      → (pair done; default & on failure)  RESET (restore-fresh) → base → RUNNING
      → (stateful pause)                    HIBERNATE → delta → … → WAKE
      → (blocking upstream call)            YIELD = hibernate(delta) + txn → WAKE
    All paths pass through RHAZARD-1..7 clearance before serving.
```

6.6 Storage tiers and run modes

```
Two orthogonal axes govern a persisted snapshot: the RECLAMATION MECHANISM (§6.5)
and the storage TIER of its delta.

Tier (where a delta lives) maps to the control plane's existing actor states:

  local (PAUSED)     the delta's objects are pinned on the node VM(s) for
                     sub-second wake if rescheduled there; the worker allocation
                     is released (compute is reclaimed) but the bytes stay local.
                     The warm-pool state.
  external (SUSPENDED) the delta lives only in object storage; no node RAM/SSD
                     reservation and no worker. The "costs nothing while idle"
                     state. Wake re-acquires through the tiered cache (§9),
                     preferring a peer/local copy before object storage.

TIER-1: The tier MUST be resolved from which CAS/tier holds the State Root's
        objects; it is reported via control-plane actor state, not a competing
        per-snapshot field (§12).
```

```
TIER-2: local↔external promotion/demotion MUST be an explicit, retryable,
        crash-safe transition: while a delta is promoting to external it remains
        validly local (resumable), local objects MUST NOT be evicted until the
        external copy is durable (WRITE-3), and a crash mid-promotion MUST leave
        the agent resumable, not stateless.

Run modes (the run_mode field, SR-1) select which planes persist across a
suspend/stop:

  clean                  boot from the base with no per-agent state (memory and
                         filesystem "none").
  golden                 start from a shared base memory snapshot (plus base
                         rootfs) with per-agent state discarded on stop; sharing
                         falls out of base copy-on-write (§5). ("golden" here names
                         this RUN MODE only; it is not an object class: the
                         "golden memory" object term is retired, §17.)
  persist-rootfs         the filesystem delta persists; memory is "none" (warm fs,
                         cold process).
  persist-rootfs+memory  the filesystem, memory, and runtime deltas persist (exact
                         process resume requires the runtime delta, RDELTA-1); the
                         flagship sub-second wake.

RUNMODE-1: run_mode is the authoritative selector of which planes a State Root
           persists; a request to discard a plane MUST NOT silently preserve it.
RUNMODE-2: run modes are orthogonal to tier (any run mode may be local or
           external) and to the reclamation mechanism.
```

6.7 Density model (informative)

```
Reclamation drives the per-agent overlay toward zero, but node-resident RAM also
includes substrate overhead that does not. The agent-overlay term:

  agent_resident = base + sum over ACTIVE agents of overlay
    reset:     completed → 0 agent overlay (no stored delta; discarded)
    hibernate: paused    → ~0 agent RAM (delta on local/external tier)
    yield:     blocked   → ~0 agent RAM (delta on tier; the proxy holds the
               in-flight call: relocated, not eliminated)

Total node-resident RAM also includes the per-sandbox runtime floor (sentry/
runtime + gofer + netstack buffers), filesystem overlay residency, proxy-held
request/response buffers, and snapshot/restore working memory. Resident
concurrency is therefore runtime-floor-bound, while TOTAL hosted agents is bounded
by ACTIVE concurrency plus delta tier capacity, so for mostly-blocked LLM agents,
reset/hibernate/yield are load-bearing for density, not merely nice to have: they
free the runtime floor, which base sharing alone cannot.
```

---

7. Startup and resume

```
Both cold start (an OCI image's first exec) and warm resume (a State Root's
restore) deliver only the bytes needed to begin useful execution, then fault the
rest. The machinery is shared: a weakly-trusted PROFILE optimizes (what to fetch
first), trusted RUNTIME REQUIREMENTS compel (what must be present), a PACK is the
transport envelope, a BOOTSTRAP carries the metadata+proofs so the critical path
needs no metadata round-trip, and COVERAGE measures readiness. Startup (cold) and
resume (warm) are duals throughout.
```

7.1 Profiles (performance hints)

```
A startup profile (cold) or resume profile (warm) is a weakly-trusted hint
recording the working set faulted earliest. Profiles optimize; they MUST NOT
compel (that is §7.2).
```

```
PROFILE-1: A profile MUST NOT weaken verification or alter any identity. It orders
           and predicts; it never authorizes.
PROFILE-2: A startup profile MUST be selected using the final command, args,
           working directory, and explicitly whitelisted env keys, never
           arbitrary user-controlled env, which MUST NOT select an expensive
           mandatory action.
PROFILE-3: A profile MUST be keyed by the identities it optimizes, plus the
           selector and a profile id, not by a mutable tag. The cold key is
           (logicalRootfsDigest, layoutDigest, selector, profile id). The warm key
           is (base-descriptor digest, State-Root digest, profile id); the
           restore working set within it is additionally keyed by base version and
           memory profile (§5.5).
PROFILE-4: Runtime-observed foreground faults MUST override profile order.
PROFILE-5: A resume profile records the pages/files a wake touches first; for the
           memory plane it is the restore working set (§5.5).

------------------------------------------------------------
```

7.2 Runtime requirements (trusted policy)

```
Runtime requirements are trusted policy derived from signed metadata, admission
policy, or operator configuration, never from an untrusted profile alone.

POLICY-1: Mandatory prehydration, fail-closed behavior, pod rejection, expected-
          mutable-file handling (§4.5), and mmap/exec-critical coverage MUST come
          from runtime requirements, not a profile.
POLICY-2: imagefsd MAY use profile hints for bounded prefetch but MUST NOT exceed
          configured byte/CPU/network/latency budgets unless trusted policy
          authorizes it.
POLICY-3: Requirements affecting fail-closed behavior MUST be covered by signature
          or attestation policy (§14).

------------------------------------------------------------
```

7.3 Packs (transport envelopes)

```
A pack, startup pack (cold) or resume pack (warm), is a transport envelope, not
a mounted object. It carries canonical block payloads that are scatter-extracted
into the canonical CAS, so pack bytes never form a separate cache identity.

PACK-1: Pack payloads MUST be addressed by canonical BlockRefs (object / level /
        blockIndex).
PACK-2: After fetching a pack, imagefsd MUST scatter-extract verified blocks into
        the canonical CAS; runtime reads MUST be served from canonical CAS keys,
        never a separate pack namespace.
PACK-3: Duplicate delivery of a block through multiple packs MUST NOT create
        duplicate cache identity or duplicate warmth credit.
PACK-4: A pack MUST be bound to the corresponding profile key (PROFILE-3): the
        cold key for a startup pack, the warm key for a resume pack.
PACK-5: A pack MUST NOT be required for correctness unless trusted policy requires
        it; absent a required pack, behavior is governed by §7.2 and the failure
        rules of §8.
PACK-6: Packs MAY be zstd-compressed; compressed bytes MUST be verified (wire
        digest) or bounded-and-sandboxed before decompression. A pack is transport,
        not an object: it carries no Terrapin identity of its own. Each block it
        delivers MUST verify against its canonical BlockRef / Terrapin proof before
        CAS admission and exposure (§8); the decompressed payload as a whole is not
        a Terrapin object.

------------------------------------------------------------
```

7.4 Bootstrap (metadata + proofs, no critical-path round trip)

```
A bootstrap is the single compact metadata record (not a Terrapin object) that
makes a start begin with zero metadata round-trips: a startup bootstrap for cold
start, a resume bootstrap for warm resume. A bootstrap MUST carry not only fetch metadata but ALL trust and
```

```
validation inputs the critical path needs, so verification itself requires no
round trip (BOOT-3).

BOOT-1: A startup bootstrap MUST carry the metadata needed to reach a mountable,
        startup-ready filesystem AND to validate it without a round trip: root
        inode and startup-critical path trie, entrypoint/loader metadata, the
        selected profile's BlockRef list, the Terrapin manifests for
        startup-required objects, the inlined proof material to verify them, and
        the trust material for the cold path's signature/policy and
        materialization-assurance checks (the signed base descriptor and its
        assurance record, §3, §14), or proof these are already locally cached.
BOOT-2: A resume bootstrap MUST carry everything needed to validate AND restore
        with no round trip: the State Root object; the runtime attestation envelope
        (§2.5); the referenced base descriptor and its builder-regime validation
        material (signature/attestation, sandbox pin, memory layout) for SR-2 /
        DIGEST-BIND-6; the memory, filesystem, and runtime delta references resolved
        to their DIRECT component objects (§6.3); the restore working set (§5.5) and
        resume
        pack BlockRef list; and inlined proofs for all resume-critical blocks. Any
        of these MAY be omitted only if proven already locally cached and verified.
        For a memory-persisting State Root whose runtime is "none", the base
        descriptor's runtime_state_policy==1 assertion (RDELTA-1, ENC-BD-5) is part of
        the carried base descriptor and MUST be validated on this no-round-trip path
        before resume proceeds.
BOOT-3: P0 startup/resume reads AND their verification MUST NOT require a metadata,
        proof, or attestation network round trip; everything needed before
        exec/unfreeze MUST be in the bootstrap or locally cached. (Proof blocks for
        an object below a configurable size SHOULD be inlined or prefetched whole;
        one 2 MiB level-1 block authenticates up to 128 GiB, §2.2.)
BOOT-4: A bootstrap SHOULD be prewarmed/seeded to candidate nodes: startup
        bootstraps to seed nodes for a rollout, resume bootstraps to candidate
        resume nodes for PAUSED actors and seed nodes for SUSPENDED actors (§10).

------------------------------------------------------------
```

7.5 Coverage

```
Coverage measures how much of a required set is present on a node, over canonical
BlockRefs (and, for cross-image base sharing, over content objects, §10).

COVERAGE-1: Coverage and scheduler warmth MUST be computed over canonical BlockRefs
            (block presence), not over packs.
COVERAGE-2: A pack receives warmth credit only for the canonical blocks it delivers
            into CAS; duplicate bytes MUST NOT inflate coverage.
COVERAGE-3: Proof-block coverage MUST be tracked separately from data-block
            coverage.
COVERAGE-4: Coverage MUST be cache-domain-aware (§9).

------------------------------------------------------------
```

7.6 Time-to-first-exec and time-to-resume

```
TTFE (cold) and TTR (warm) are the primary latency SLOs and MUST be broken into
components so operators can attribute a slow start.

  TTFE = image resolve + metadata/bootstrap resolve + signature/policy verify
       + materialization-assurance verify + profile select + startup pack fetch
       + scatter/verify + mount + runtime create + exec
  TTR  = State Root resolve + runtime-attestation verify + base resolve (warm via
       the base plane) + memory, filesystem & runtime delta resolve + verify + resume
       pack fetch (hot working set) + restore (page-in; base runtime-state + runtime
       delta applied) + overlay attach + unfreeze + first request

TTFE-1: Metrics MUST decompose TTFE and TTR into the components above, so a start
        can be attributed (metadata-, proof-, network-, decompression-,
        verification-, restore-, policy-, or runtime-bound) (§13).
TTR-1: A resume MUST fault the hot working set first and lazily fault the cold
       tail at P0 (§5.4, §8); full hydration MUST NOT be on the critical path
       unless run mode or policy requires it. If the runtime restore eagerly
       touches the whole memory image, the start MUST be reported as restore-bound,
```

```
       not as a thin lazy resume. The runtime delta is resolved directly (RDELTA-4)
       and applied with the base runtime-state before unfreeze (RDELTA-5); it is part
       of restore, not lazily faulted.

------------------------------------------------------------
```

7.7 Profile drift and materialization strictness

```
DRIFT-1: imagefsd MUST report profile misses (P0 startup/resume faults not covered
         by the selected pack), by phase.
DRIFT-2: A profile SHOULD be marked stale when its miss ratio exceeds a configurable
         threshold (default 20% over a meaningful window) and SHOULD trigger a
         re-profiling event; a new profile MUST use a new profile identifier and
         MUST NOT mutate the old profile record. (Profiles and runtime
         requirements are signed metadata records: their trust identity is the
         signature/digest over their bytes, §14, not Terrapin object classes.)

Materialization strictness selects how much must be present before a milestone;
it is trusted policy and is orthogonal to run mode (§6.6) and tier:

  startup-only        only exec/startup-critical present before exec
  readiness-critical  exec + readiness present before readiness
  full-before-ready   full hydration before readiness
  full-before-exec    full hydration before exec
  lazy-risk-accepted  runtime faults on cold blocks allowed by policy

STRICT-1: A deployment MUST be able to choose a materialization strictness mode.
STRICT-2: Strictness mode MUST be visible in status, events, and metrics; runtime
          lazy-materialization failures MUST be reported with clarity equivalent
          to a classic image-pull failure (§13).

------------------------------------------------------------
```

8. Fetch, verification, and transport

```
Every byte exposed to a workload is verified against a trusted Terrapin identifier
(§2). This section is the read path beneath that guarantee: how a block is
located, deduplicated in flight, prioritized, hedged, and sourced (from local
cache, peers, mirrors, or origin) without ever trusting transport.

------------------------------------------------------------
```

8.1 Read path

```
  cache hit:  read → map (path/offset or guest address) → BlockRef → local CAS
              hit (already verified; local integrity per READ-7) → return bytes.
  cache miss: read → BlockRef → join/create singleflight → P0 fetch → source
              select → fetch (compressed or raw) → verify the wire digest if
              present → bounded decompression if needed → load proofs (from
              bootstrap/cache) → verify the extracted block against its canonical
              BlockRef / Terrapin proof → admit to canonical CAS → return bytes.
              (Uncompressed-identity verification can only occur AFTER
              decompression; compressed bytes are verified beforehand only against
              a wire digest, COMP-2/COMP-3.)

READ-1: Foreground reads MUST have strict priority over background hydration.
READ-2: Bytes MUST NOT be returned until verified against a trusted Terrapin
        identifier (§2.2 TERRAPIN-4); this holds for filesystem blocks, memory
        blocks, and proof blocks alike.
READ-3: imagefsd MUST coalesce concurrent requests for the same BlockRef unless
        intentionally hedging (§8.4).
READ-4: Adjacent readahead SHOULD occur only when it does not violate foreground
        latency budgets.
READ-5: Per-container, per-tenant, and per-node in-flight fetch memory MUST be
        bounded.
READ-6: A failed source fetch MUST fall back to other allowed sources; a
        verification failure MUST reject the source and retry elsewhere if policy
```

```
        allows (§8.6, §14).
READ-7: A local CAS entry is treated as verified after admission (READ-2), so the
        node's local store (RAM CAS, page cache, on-disk CAS) is inside the node's
        trusted computing base for already-admitted blocks. A deployment MUST protect
        that store against post-admission corruption (disk/bit rot, stale page cache,
        buggy local mutation) by at least one mechanism: a per-block integrity check
        on read, fs-verity/dm-verity over the CAS backing, or periodic scrubbing
        against the Terrapin tree. On detected corruption the entry MUST be treated
        as ABSENT and re-fetched and re-verified (§8.2), never served. (Cross-host
        transport is never a trust anchor, TERRAPIN-4; READ-7 is about the LOCAL
        store after admission.) The mechanism is an operational choice, but one MUST
        be stated and enabled.

------------------------------------------------------------
```

8.2 Block singleflight

```
Terrapin verifies 2 MiB blocks; a single sequential read may shatter into many
smaller VFS/fault requests. A per-BlockRef singleflight state machine collapses
them.

  block lifecycle: ABSENT → INFLIGHT → VERIFIED → (EVICTED) ; → FAILED

BLOCK-1: imagefsd MUST maintain a single in-flight operation per BlockRef.
BLOCK-2: Multiple reads/faults for ranges within the same block MUST join the same
         in-flight operation.
BLOCK-3: The first miss MAY trigger a full 2 MiB fetch; later reads MUST NOT
         trigger duplicate range requests unless intentionally hedging (§8.4).
BLOCK-4: Once verified, all waiters MAY be satisfied from RAM CAS or page cache.
BLOCK-5: Singleflight state MUST be cache-domain-aware (§9).

------------------------------------------------------------
```

8.3 Fetch priority

```
  P0  synchronous foreground read or page fault (workload is blocked)
  P1  predicted next foreground read (strong evidence: sequential, mmap/ELF
      locality, recent trace, profile)
  P2  selected profile / pack critical block
  P3  readiness-path prefetch
  P4  background hydration
  P5  full-image archival / prewarm

QUEUE-1: P0 MUST preempt P2-P5.
QUEUE-2: P1 SHOULD be generated only from strong evidence.
QUEUE-3: P4/P5 MUST be rate-limited during cluster-wide rollout.
QUEUE-4: Fetch admission SHOULD consider NIC pressure, disk-queue depth, registry
         rate limits, peer health, cache-domain policy, and origin breaker budgets.
QUEUE-5: Background hydration MUST be interruptible.

------------------------------------------------------------
```

8.4 Hedged requests and origin circuit breakers

```
Hedging cuts individual fault tail latency; origin breakers stop hedges from
stampeding a degraded origin during a correlated rollout. Both are required.

HEDGE-1: P0 requests MUST support hedged fetch; a hedge SHOULD issue when the first
         source exceeds min(configured hedge delay, remaining fault budget / 4).
HEDGE-2: The first VERIFIED response wins; losing hedges SHOULD be cancelled
         immediately.
HEDGE-3: A corrupt response MUST penalize or eject the source; a slow response
         MUST lower its score but MUST NOT be treated as corrupt.
HEDGE-4: P2-P5 background hydration SHOULD NOT hedge by default.
HEDGE-5: Hedging MUST respect tenant, registry-auth, egress, and cache-domain
         policy.
ORIGIN-1: P0 origin hedging MUST be protected by per-registry and per-image circuit
          breakers and a token-bucket (or equivalent) origin-hedge budget.
```

```
ORIGIN-2: When the origin breaker is open, P0 hedges MAY target local mirrors or
          peers but MUST NOT stampede origin unless emergency policy allows it.
ORIGIN-3: The controller SHOULD coordinate origin-hedge budgets cluster-wide during
          rollouts; origin-hedge denials MUST be visible in telemetry (§13).
```

8.5 Local cache hierarchy and source selection

```
Recommended fetch order: process page cache → imagefsd RAM CAS → local SSD/NVMe
CAS → same-host cache → same-rack/AZ peer → node-local registry mirror → regional
mirror/CDN → origin registry.

SRC-1: Foreground reads MUST bypass or preempt background hydration at every tier.
SRC-2: For P0, source selection SHOULD optimize expected completion time
        (queue delay + tail latency + verification-failure + cross-AZ +
        auth-refresh + origin-breaker penalties), not raw bandwidth; background
        hydration MAY prefer high throughput.
SRC-3: Startup/mmap-critical and proof blocks SHOULD be retained preferentially in
        cache (§9).
SRC-4: Cache entries MUST be promoted atomically only after verification, and keyed
        in the internal digest plane with cache-domain separation (§9).
```

8.6 Peer protocol

```
Peers accelerate fan-out but are never trusted for integrity. Peers exchange
Terrapin BlockRefs and proof blocks, not OCI blobs.

PEER-1: Peers MUST NOT be trusted for integrity; receivers MUST verify every block
        locally before exposure (READ-2).
PEER-2: Peers MAY refuse service on load/tenant/network pressure and MUST apply
        rate limits to avoid cascading failure.
PEER-3: Cross-cache-domain Have()/FetchBlock() MUST be denied unless policy
        explicitly allows it (§9); registry credentials MUST never be sent to
        peers.
PEER-4: Corrupt peer responses MUST eject/penalize the peer and MUST be observable
        (§13).
PEER-5: P2P is not required for correctness; the protocol is defined so it can be
        added without changing internal identity (phasing is §16).
```

8.7 Registry and mirror interaction

```
LLIFS uses OCI registries for external storage and discovery: registry-facing
operations use OCI sha256 descriptors; internal verification uses terrapin-sha256.

REG-1: OCI descriptor verification MUST use the external digest plane; filesystem/
       memory-byte verification MUST use the internal plane.
REG-2: LLIFS MUST support origin-registry fallback unless policy forbids it, and
       SHOULD prefer mirrors / pull-through caches near the cluster.
REG-3: LLIFS SHOULD use HTTP range requests where supported and MUST preserve
       registry authentication boundaries.
REG-4: LLIFS SHOULD discover metadata via OCI Referrers, and MUST support a
       tag-fallback discovery mode for registries without Referrers; tag-fallback
       records MUST still bind to the OCI image digest internally, and signature/
       trust verification MUST be identical under both.
```

8.8 Compression and decompressor hardening

```
LLIFS distinguishes filesystem/memory identity (uncompressed) from wire identity
(compressed).
```

```
COMP-1: An LLIFS object's identity MUST be the Terrapin digest of its uncompressed
        canonical bytes (§2.1 DIGEST-2, §4, §5). A wire (compressed) digest is
        transport-only and MAY additionally be carried; it MUST NOT be used as
        object identity.
COMP-2: If a wire digest is available, compressed bytes MUST be verified before
        decompression; otherwise decompression MUST enforce bounded compressed
        input, output, memory, CPU/time, and expected length, in an isolated
        decompressor.
COMP-3: Decompressed output MUST be verified against the uncompressed identity (per
        block / per object) before exposure; unverified decompressed bytes MUST
        NEVER reach VFS or guest memory.
COMP-4: Decompression, archive parsing, and untrusted-metadata parsing SHOULD run
        in isolated workers with seccomp, rlimits, no ambient network, no registry
        credentials, and a minimal filesystem view.
```

8.9 Failure behavior

```
FAIL-1: If a foreground (P0) block cannot be fetched-and-verified from any allowed
        source, the read/fault MUST fail (and, for a memory fault, surface as the
        process-fatal-class event of COW-7), never hang silently or expose zeros.
FAIL-2: Repeated verification failure for one object SHOULD trigger security
        alerting (§14).
FAIL-3: If imagefsd crashes, active mounts SHOULD recover from verified cache state
        or fail closed.
FAIL-4: Failures that would classically appear as ImagePull errors MUST be reported
        with equivalent clarity when they occur during runtime lazy
        materialization (§13, STRICT-2).
```

---

9. Cache domains, leases, and garbage collection

```
The fetch tiers and source order are §8.5. This section governs WHO may share a
cached block (cache domains and keying), how shared objects are kept alive
(leases and liveness), and how they are reclaimed (GC), at agent-FaaS scale,
where one base object is referenced by a million agents.
```

9.1 Cache domains

```
A cache domain is the maximum scope within which content-presence timing leakage
is acceptable (§2, §17). It is the isolation unit for dedup, keying, warmth, and
GC.

DOMAIN-1: A cache domain has a canonical ID: a length-prefixed raw byte string
          derived from a configured source (cluster | node-pool | namespace |
          tenant | workload-identity | enclave-group). The ID is opaque bytes; no
          Unicode normalization.
DOMAIN-2: The domain source MUST be configured per deployment and MUST be
          deterministic: the same workload context maps to the same domain ID.
DOMAIN-3: Domain assignment MUST be trusted (admission/snapshotter from the
          workload's namespace/tenant/identity), never workload-controlled. A
          workload MUST NOT choose its own domain to reach another's cache.

Keying (two modes; §8.5 SRC-4):

  trusted-shared:  cache_key = terrapin-sha256:<object> / level / blockIndex
  private-domain:  cache_key = HMAC(domainKey[epoch], <object> / level / blockIndex)

KEY-1: In private-domain mode, physical cache names MUST be the HMAC of the
       canonical BlockRef under a per-domain secret, so content presence is not
       inferable across domains by probing cache names.
KEY-2: The per-domain secret MUST be node-local and MUST NOT be sent to peers or
```

```
       registries (PEER-3).
KEY-3: Keys are VERSIONED by epoch. Rotation bumps the epoch and writes new entries
       under the new key; rotation MUST NOT delete or rewrite existing entries
       (avoiding a mass invalidation storm); old-epoch entries become
       unreferenced and are reclaimed by GC.
KEY-4: A bounded grace window MAY keep the prior epoch readable. A prior-epoch
       entry is GC-eligible ONLY when it has no live lease (LEASE-3); an old-epoch
       entry that still has a live lease MUST remain readable, or be lazily
       re-written under the current epoch, before the grace window expires, so
       rotation never strands live-leased content (rotation MUST NOT reduce
       reachability of any leased object).
KEY-5: Proof-block caches obey the same domain keying as data blocks. Shared proof
       caching MAY be enabled only under the same cache-domain policy gate as
       cross-domain content sharing (SHARE-1, REDACT-3), because proof-block
       presence also reveals object presence.

------------------------------------------------------------
```

9.2 Leases and liveness

```
Dedup means one content object, base object, or delta may be referenced by many
images, profiles, or agents within a domain.

Lease types: runtime, startup, profile, peer, operator pin, cache-domain,
materialization-attestation, pause (pins a PAUSED actor's objects on the local
tier), and state-root (pins a SUSPENDED State Root's objects durably without a
node pin).

LEASE-1: The substrate MUST maintain liveness per (cache domain, object).
         Acquiring any referencing lease pins the object; releasing unpins it.
LEASE-2: Lease updates MUST be atomic and crash-safe: a crash MUST NOT
         leak a reference (pinning forever) nor drop one (evicting in-use
         content). Liveness MUST be reconstructable after a crash from a durable
         journal or by rescanning the live lease set, so neither leaked nor dropped
         pins survive recovery.
LEASE-3: An object with a live lease in a domain MUST NOT be evicted from that
         domain; liveness aggregates across ALL referencing images/agents;
         eviction is a domain-level decision, not per-image.
LEASE-4: Hibernation deltas are leased objects: a hibernated agent holds a lease on
         its delta, and the base it will wake to MUST be kept live by a lease that
         reflects every active AND hibernated agent (§6 HIBER-6).

------------------------------------------------------------
```

9.3 Garbage collection

```
GC-1: GC MUST be digest-safe and crash-safe: partial objects written under
      temporary names, atomically promoted only after verification, never deleting
      an object with a live lease (§6 WRITE-2/WRITE-4).
GC-2: GC MUST respect cache-domain boundaries; an object's eligibility is evaluated
      per domain by that domain's liveness and epoch.
GC-3: Eviction order SHOULD prefer unreferenced, low-fan-in, cold objects, and
      preferentially retain high-fan-in objects (shared by many images/agents) and
      startup/mmap-critical/proof/base blocks; evicting a high-fan-in object harms
      many workloads at once.
GC-4: Stale-epoch entries (post-rotation, past the grace window) with no live lease
      are first-class GC candidates.
GC-5: GC SHOULD export eviction reasons (§13).

Fleet-scale GC (a shared base object may have ~a million referrers, so a hot
per-agent refcount on it is impractical). GC is ONE model: a mark-sweep whose ROOTS
are the live leases (§9.2) and the live State Roots; an object survives iff it is
reachable from a root. Leases and State Roots are roots/pins for that sweep; this
reconciles LEASE-3 (a leased object is never evicted) with GC-FLEET (no hot
per-operation refcount): "not per-operation refcount" means the base is not
incremented/decremented per agent, NOT that leases are ignored.

GC-FLEET-1: Shared base objects (base rootfs content objects + the base memory
            snapshot, including the snapshot used by the golden run mode) MUST be
            pinned by a template/operator lease while the template is live and MUST
```

```
            NOT be GC'd by per-agent churn; this avoids a hot per-agent refcount on
            the base.
GC-FLEET-2: Per-agent objects (memory, filesystem, and runtime delta components) are
            retained
            iff REACHABLE from a live actor's retained State Root or a live lease,
            plus lineage retention/squash (§6.3), not by a per-object refcount.
GC-FLEET-3: The mark-sweep from each live State Root MUST traverse its base
            descriptor reference, so a base stays live while ANY retained State Root
            (active or hibernated) can resume from it, even if the template/
            operator lease has been removed (resolving LEASE-4 vs GC-FLEET-1). The
            sweep MUST NOT use per-operation refcounting and MUST respect
            cache-domain boundaries (GC-2).
GC-FLEET-4: Lineage retention policy MAY squash intermediate State Roots in a
            suspend chain (merging deltas into a later State Root); squashing MUST
            preserve the resumability of retained State Roots and the
            self-contained direct references of §6.3.
```

9.4 Multi-tenancy and redaction

```
REDACT-1: Metrics exposed to a tenant MUST be scoped to that tenant's domain and
          MUST NOT reveal other domains' image presence, cache warmth, peer
          availability, or content-presence timing.
REDACT-2: Cross-domain aggregates (total cache size, global warmth, dedup ratios)
          MAY be exposed only to the operator/cluster role.
REDACT-3: Base-warmth and content-presence signals used for scheduling (§10) MUST
          be domain-scoped in any tenant-visible surface; cross-domain warmth is
          operator-only.
```

9.5 Cross-domain sharing (opt-in)

```
SHARE-1: Cross-domain content sharing (trusted-shared mode, unkeyed) MUST be
         explicit opt-in; the default is private-domain (keyed). Cross-domain
         Have()/FetchBlock() are denied unless policy allows (PEER-3).
SHARE-2: A deployment MAY define a trust hierarchy (e.g. share within a tenant's
         node-pool but not across tenants) by choice of the domain ID source
         (DOMAIN-1); sharing is within a domain by construction.
```

---

10. Scheduler integration and warmth

```
Keeping the cluster scheduler out of the hot path is a core thesis: placement is
guided by WARMTH (what verified state a node already holds) so a start or resume
faults few bytes. Warmth is computed over canonical BlockRefs and content objects
(§7.5), is cache-domain-scoped (§9), and is exposed to scheduling as a score.
```

10.1 Warmth model

```
WARM-1: Warmth MUST be computed over canonical BlockRefs (block presence) and, for
        cross-image/cross-agent sharing, over content objects and base objects,
        never over packs (COVERAGE-1/2).
WARM-2: A node MUST be able to advertise BASE warmth (the set of held content
        objects and base memory snapshot blocks) scoped to a cache domain and
        INDEPENDENT of any specific image or agent. At agent-FaaS scale base
        warmth is the dominant placement signal: a node warm on a template's base
        can run almost any of that template's agents.
WARM-3: Proof-block warmth MUST be tracked separately from data-block warmth.
WARM-4: Per-image / per-profile coverage (§7.5) MUST still be tracked; base warmth
        is additive context, not a replacement.
```

```
WARM-5: All warmth advertisements MUST be cache-domain scoped and MUST NOT reveal
        presence across domains (REDACT-3); cross-domain warmth is operator-only.

------------------------------------------------------------
```

10.2 Node score

```
A node's score for placing a start/resume SHOULD combine the warmth that reduces
its critical path, minus pressure that raises it:

  score = w1·startup/resume-critical blocks present
        + w2·proof blocks present
        + w3·mmap/exec-critical present
        + w4·base warmth (content objects + base memory snapshot)
        + w5·readiness blocks present
        + w6·peer locality
        - w7·disk pressure - w8·network pressure - w9·origin-breaker risk

SCORE-1: All WARMTH inputs to the score MUST be cache-domain-scoped (WARM-5). The
         node-pressure and origin-breaker-risk terms are permitted non-warmth
         inputs and MUST themselves be domain-safe (they MUST NOT reveal another
         domain's content presence).
SCORE-2: Weights are deployment-tunable; the scheduler SHOULD prefer placing
         latency-sensitive workloads on nodes with high selected-profile and base
         warmth.

------------------------------------------------------------
```

10.3 Resume affinity

```
AFFINITY-1: For a PAUSED actor (local tier, §6.6) the scheduler SHOULD prefer the
            node(s) that still hold the actor's local delta and base resident, for
            sub-second resume; this is the warm-pool placement signal.
AFFINITY-2: For a SUSPENDED actor (external tier) the scheduler SHOULD prefer any
            node warm on the actor's base (WARM-2), since the per-agent delta is
            small and faults lazily (§7.6).
AFFINITY-3: Resume affinity MUST be advisory: a State Root MUST be resumable on any
            node that can validate it and satisfy the sandbox pin (§2.5), so loss
            of a warm node never strands an actor; it only costs latency.

------------------------------------------------------------
```

10.4 Rollout and fan-out

```
ROLLOUT-1: For a large rollout, the controller SHOULD resolve image/metadata once
           per cluster/region, select seed nodes per rack/AZ/cache domain, and
           prewarm the bootstrap and pack to seed nodes before scheduling pods onto
           startup-warm nodes (§7.4 BOOT-4).
ROLLOUT-2: A FORK (§6.3) is an intra-cluster rollout of one snapshot to N children;
           it SHOULD reuse the seed/prewarm/hedge machinery (FORK-4), seeding the
           source's resume bootstrap and hot blocks to the children's nodes.
ROLLOUT-3: Background prewarm/hydration MUST be interruptible and rate-limited
           (QUEUE-3/QUEUE-5) and MUST respect origin-breaker budgets (§8.4).

------------------------------------------------------------
```

10.5 Warmth state

```
WARM-6: Warmth state exposed for scheduling (per node: startup/proof/mmap/
        readiness/base coverage ratios, last-verified time) MUST be keyed by the
        identities it describes: the PROFILE-3 cold key (logicalRootfsDigest,
        layoutDigest, selector, profile id) for images; the base descriptor digest
        for bases, within a cache domain, and is carried by the control-plane
        integration (§12), not by a workload-visible surface (REDACT-1).

------------------------------------------------------------
```

11. Runtime adapters

LLIFS is one verified, content-addressed store; runtimes are ADAPTERS over it.
Internal identity (Terrapin) and the verify-before-expose invariant do not change
per adapter. gVisor (runsc) is the primary target; Firecracker microVM is next.

11.1 Adapter model

ADAPT-1: A runtime adapter MUST source filesystem and memory bytes from the
verified CAS (§8) and MUST NOT expose any byte before it is verified
against a trusted Terrapin identifier (§2.2 TERRAPIN-4, §5 MVERIFY-1).
ADAPT-2: An adapter MUST NOT alter object identity, the binding chain (§2.5), or
the restore-hazard obligations (§5.6); these are runtime-independent.
ADAPT-3: An adapter MUST honor the sandbox pin (§2.5): a snapshot is restorable
only under a runtime compatible with the pinned, per-architecture sandbox
assets, and the base compatibility matrix (§11.4).

11.2 gVisor (primary)

Most of the memory plane is reused, not built; one capability is net-new.

GVISOR-1: The filesystem plane SHOULD be served to the sandbox kernel (Sentry)
through a CAS-backed lisafs gofer, with no host-kernel FUSE on the read
path. Bytes are verified at the serving layer before they reach the
guest. This needs no Sentry change.
GVISOR-2: Lazy/on-demand restore SHOULD be REUSED, not rebuilt: the existing
background-restore mode (the runtime starts the workload after kernel
state loads and faults remaining pages on demand) is the cold-tail path.
The page SOURCE is redirected by serving the runtime's pages file from
the same CAS-backed gofer, where bytes are verified as served. The gofer
serves the runtime's expected page view by mapping the runtime's
pages-file offsets to LLIFS BlockRefs through the committed memory layout
(§5.3): the base memory snapshot identity is unchanged; the pages file
is a view, not a new object. (If the runtime's native pages-file layout
is not already guest-address-linear, the memory layout provides the
deterministic mapping.) Lazy, remote-sourced, and verified restore
therefore need no Sentry change.
GVISOR-3: The ONE net-new capability is SHARED COPY-ON-WRITE BASE restore: backing
each restored sandbox's base guest-memory region with a MAP_PRIVATE
mapping of one shared, verified base memory snapshot so resident base
pages are physically shared across sandboxes and only dirty pages are
private (§5.4). The runtime's memory-file backing MUST be extended to
support mapping each sandbox's base region onto a shared, externally
populated, verified base file (today that backing is not pluggable).
Without this, every restore materializes its own multi-GiB guest memory
and the density thesis fails; with it, a million agents share one base.
This is the substrate's sole hard runtime dependency and its central
density mechanism. (Justification: §6.7. It is a minimal, surgical change:
it changes where base pages are backed, not the checkpoint format.)
GVISOR-4: The runtime-state blob (§5.1) MUST be compatible with the runtime's
platform mode; the mode MUST be recorded in the base descriptor and the
compatibility matrix (§11.4).
GVISOR-5: gVisor directfs (sandbox-direct host-fd filesystem access that bypasses the
gofer round trip) MUST be DISABLED for the LLIFS lower in v1: the gofer is
the verify-before-expose and cache-domain enforcement choke point
(GVISOR-1, §9), and directfs would let the sandbox read lower bytes the
gofer never verified. directfs MAY be enabled for the LLIFS lower only once
the native path preserves the identical verify-before-expose
(DIGEST-BIND-6) and cache-domain (§9) semantics (a §11.5 / Phase 4
evaluation), and MUST NOT be enabled merely for performance.
GVISOR-6: The byte paths are distinct abstractions even where they share code: rootfs
path/offset reads use the CAS-backed lisafs gofer (GVISOR-1); restore page
reads use a CAS-backed PAGE PROVIDER (the pages-file view of GVISOR-2);
shared base residency is the MemoryFile/shared-backing change (GVISOR-3). A
userfaultfd MISSING fault MUST populate the shared verified base backing,
not a per-sandbox private copy (COW-6a); once a base range is resident,

additional sandboxes MUST resolve it by shared/minor-fault continuation,
never by repeating a private copy.

11.3 Firecracker microVM (secondary)

FC-1: The filesystem plane SHOULD be served via virtiofs backed by the verified
CAS.
FC-2: The memory plane SHOULD use the microVM snapshot/restore path with a
userfaultfd handler that serves verified base pages from the CAS, sharing
resident pages across microVMs and copy-on-writing per-VM dirty pages (the
same §5.4 model).
FC-3: Firecracker and gVisor adapters MUST address the same base memory snapshot by
the same Terrapin identifier; a base captured for one runtime MAY be
incompatible with another (§11.4), but the object identity and verification
path are identical.

11.4 Base compatibility matrix

A base memory snapshot is restorable only by a compatible runtime. The base
descriptor MUST commit a compatibility matrix; a node MUST NOT restore a base it
cannot satisfy.

MATRIX-1: The matrix MUST record: runtime and runtime version; platform mode;
CPU architecture and page size; CPU feature mask (a base captured using
features absent on the target MUST NOT restore there); checkpoint/state
format version; sandbox-kernel/guest ABI expectations; and the seccomp/
cgroup/namespace assumptions at capture.
MATRIX-2: A mismatch on any required field MUST fail closed with a distinct event
(§13), never a silent or best-effort restore. The sandbox pin (§2.5) and
the matrix together gate placement (§10 AFFINITY-3).
MATRIX-3: Compatibility is decided by a named verifier step, MATCH-COMPAT(committed,
restorer), where the COMMITTED value records what the producer captured and
the RESTORER value is the candidate node's capability: runtime_id and arch
MUST be equal; platform_mode and page_size MUST be equal; runtime_version
and checkpoint/state-format version MUST satisfy the runtime's declared
backward-compat rule (default: equal); the CPU feature mask MUST be a
superset (the restorer provides every committed feature); and the seccomp/
cgroup/namespace and ABI assumptions MUST be satisfiable. MATCH-COMPAT MUST
run before any base or runtime-delta byte is applied (RDELTA-2/5); any
unsatisfied field fails closed (MATRIX-2).

11.5 Native verification path (later)

A native, page-cache-integrated backend MAY later reduce per-byte overhead; it
MUST NOT weaken the verification or identity model.

NATIVE-1: A native backend (e.g. EROFS/fscache, composefs-style verified trees,
fs-verity, virtiofs) MUST preserve the logical rootfs and layout binding
and the base memory snapshot identity.
NATIVE-2: A native backend MUST either verify with Terrapin directly or bind its
native verification root (e.g. an fs-verity root) into signed LLIFS
metadata, so Terrapin secures network/remote identity while the native
mechanism secures local page-cache/disk reads, without double-paying
verification on the critical path.
NATIVE-3: Any native backend MUST preserve cache-domain policy (§9).

---

12. Control-plane binding

LLIFS is a substrate; the agent control plane is its consumer adapter, exactly as containerd/Kubernetes is a consumer adapter for the filesystem plane. This section binds LLIFS to the control plane's existing data model (the ateapi/atelet/ateom protos) rather than introducing a parallel one. Proto additions below are PROPOSED (not the current schema) and are required deliverables, landing in Phase 2 with hibernate/resume (§16).

12.1 Mapping to the existing model

```
  State Root (§6.2)            ≈ the snapshot's self-describing manifest, carried by
                               Actor.latest_snapshot_info (12.2).
  Actor.Status                 the lifecycle (§6.5/§6.6): RUNNING, SUSPENDING/
                               SUSPENDED (external tier), PAUSING/PAUSED (local
                               tier), RESUMING.
  LocalSnapshotInfo.node_vms_with_local_snapshots
                               the warm-node placement index for PAUSED deltas
                               (§10.3); a single source of truth (12.2).
  atelet.SandboxAssets / AssetFile.sha256 (node-control-plane material, not
   ateapi.Actor state)
                               the per-architecture sandbox pin (§2.5); the runtime
                               binaries content-addressed in the external plane.
  ResumeActorRequest.boot      the skip-golden / boot-from-scratch control selecting
                               clean vs golden run mode (§6.6); run_mode persistence
                               is the State Root's field.
  SuspendActor / PauseActor / ResumeActor (control)
   → Checkpoint / Restore (atelet) → ateom
                               drive the write path (§6.4) and resume path (§7);
                               imagefsd is the node-local layer beneath them (12.4).
```

12.2 SnapshotInfo migration

Today ateapi.SnapshotInfo carries a URI prefix to an opaque snapshot (ExternalSnapshotInfo.snapshot_uri_prefix; LocalSnapshotInfo.snapshot_prefix), with only external=2 and local=3 in the oneof. The locked decision is a backward-compatible oneof extension to a content-addressed, lineage-bearing State Root, not a forklift:

```
  // PROPOSED addition to ateapi.proto (state_root = 4 is new):
  message SnapshotInfo {
    SnapshotType type = 1;
    oneof data {
      ExternalSnapshotInfo  external   = 2;  // legacy opaque blob (object store)
      LocalSnapshotInfo     local      = 3;  // legacy opaque blob (node VM)
      StateRootSnapshotInfo state_root = 4;  // NEW: content-addressed State Root
    }
  }
  message SnapshotPlacement {
    // warm-node list only; same semantics as
    // LocalSnapshotInfo.node_vms_with_local_snapshots (single source of truth).
    repeated string node_vms_with_local_snapshots = 1;
  }
  message StateRootSnapshotInfo {
    string state_root_digest = 1;   // canonical terrapin-sha256:<hex> of the
                                    // State Root (§2.1, §6.2)
    SnapshotPlacement placement = 2; // warm-node placement only; tier is CAS state
  }
```

CP-1: A State Root snapshot MUST be referenced through StateRootSnapshotInfo carrying the State Root identifier; legacy external/local remain valid for opaque snapshots during migration. The proto additions are a REQUIRED Phase 2 deliverable (§16), landing with hibernate/resume; LLIFS cannot publish/resume a State Root through

```
      Actor.latest_snapshot_info without them.
CP-2: Tier (local/external) MUST be resolved from which CAS/tier holds the State
      Root's objects (§6.6 TIER-1), consistent with SnapshotType/CheckpointType;
      SnapshotInfo.type is informational/ignored for the state_root arm and MUST
      NOT be a second tier authority.
CP-3: Lineage (parent) is NOT duplicated in the proto; it lives only in the State
      Root (§6.2). The control plane MAY cache parent for queries but MUST treat
      the State Root as authoritative.
CP-4: The warm-node placement list MUST have a single representation
      (SnapshotPlacement, reusing the legacy field's semantics); implementations
      MUST NOT mirror it across competing fields.
CP-5: in_progress_snapshot semantics MUST be preserved: a State Root MUST NOT be
      published into latest_snapshot_info until the write path (§6.4 WRITE-3) has
      durably promoted every referenced object, so a crash mid-suspend leaves the
      prior State Root resumable.

------------------------------------------------------------
```

12.3 Fork binding

```
Fork (§6.3) is net-new to the data model: CreateActor today derives only from a
template. A create-from-snapshot needs a source-snapshot field:

  // PROPOSED addition to ateapi.CreateActorRequest:
  string from_state_root = 5;  // source State Root id (terrapin-sha256:<hex>);
                               // empty = derive from template. Field number 5 MUST
                               // be reserved for this addition once accepted.

CP-6: When from_state_root is set, CreateActor MUST allocate a NEW actor and the
      child's genesis State Root MUST set parent = from_state_root (§6.3 FORK-1).
      The source State Root is authoritative for base, memory delta, filesystem
      delta, runtime delta, sandbox pin, and workload identity; template fields, if
      supplied, MUST
      be omitted or match and MUST NOT override inherited state; worker_selector
      applies only as a child placement override.

------------------------------------------------------------
```

12.4 Checkpoint/Restore binding

```
The atelet/ateom RPCs today carry only opaque checkpoint references: atelet
Checkpoint/Restore have a CheckpointType plus a local/external configuration
oneof; ateom CheckpointWorkload/RestoreWorkload carry only an object-storage
snapshot_uri_prefix. None carry a State Root, delta, or plane fields, so they
cannot drive the §6 write / §7 resume path directly. Two conforming options:

CP-7: Either (a) extend these RPCs with State-Root- and delta-capable request/
      response fields, or (b) front them with a node-local adapter in imagefsd
      that, on Checkpoint, ingests the runtime's produced checkpoint into the memory,
      filesystem, and runtime deltas and emits a State Root (§6.4, DELTA-2R), and on
      Restore resolves a State Root into the page view, the base runtime-state, and
     the runtime delta the RPC expects (§11.2, RDELTA-5). Until the RPCs carry State Roots, the adapter (b) MUST be provided;
      neither path may weaken verification (§8) or the freeze-epoch guarantee
      (§6.4 WRITE-1). These RPC/proto extensions are a REQUIRED Phase 2 deliverable
      (§16), landing with hibernate/resume.

------------------------------------------------------------
```

12.5 Run-mode reporting

```
CP-8: The selected run mode MUST be observable from Actor state/metadata (not the
      Actor.Status enum, which it cannot carry), resolved from the State Root via
      Actor.latest_snapshot_info → the StateRootSnapshotInfo
      state_root_digest → the State Root's run_mode (§6.2). No separate
      Actor.run_mode field is
      required; a control plane MAY surface a denormalized copy but the State Root
      is authoritative.
```

---

13. Observability

LLIFS treats latency and runtime-materialization failures as primary observables.
The two headline SLOs, TTFE (cold) and TTR (warm), MUST be decomposable into
components (§7.6) so a slow start is attributable rather than opaque.

13.1 Requirements

OBS-1: Metrics MUST decompose TTFE and TTR into their components (§7.6) so a start
can be attributed as metadata-, proof-, network-, decompression-,
verification-, restore-, copy_up-, mmap-, policy-, or runtime-bound.
OBS-2: Runtime lazy-materialization failures MUST be reported with clarity
equivalent to a classic ImagePull failure (§8 FAIL-4); the materialization
strictness mode (§7.7) MUST be visible in status, events, and metrics.
OBS-3: A resume that is restore-bound (the runtime touched the whole memory image,
§7.6 TTR-1) MUST be reported as such, not as a thin lazy resume.
OBS-4: Repeated verification failures and corrupt-source ejections MUST surface as
security events (§14), distinct from ordinary fetch failures.
OBS-5: All tenant-visible telemetry MUST be cache-domain-scoped (§9 REDACT-1..3);
cross-domain aggregates are operator-only.
OBS-9: Runtime-delta cost MUST be reported separately from memory- and
filesystem-delta cost (§13.2), because the runtime delta is resolved and
applied on the resume critical path (RDELTA-4/5) and could become the restore
bottleneck. A deployment SHOULD set a runtime-delta size budget; a runtime
delta exceeding it MUST be reported (llifs_runtime_delta_oversized_total) and
MAY be rejected at capture by policy.

13.2 Metrics (recommended)

```
  Latency / SLO:
    llifs_time_to_first_exec_ms          llifs_time_to_resume_ms
    llifs_mount_latency_ms               llifs_foreground_fault_latency_ms
    llifs_memory_fault_latency_ms        llifs_mmap_fault_latency_ms
    llifs_time_to_ready_ms
  Verification / proofs:
    llifs_terrapin_block_verify_latency_ms   llifs_terrapin_verify_failures_total
    llifs_terrapin_proof_cache_hits_total
  Snapshot write / lifecycle:
    llifs_snapshot_write_latency_ms      llifs_snapshot_delta_bytes
    llifs_memory_delta_bytes             llifs_filesystem_delta_bytes
    llifs_runtime_delta_bytes            llifs_runtime_delta_apply_latency_ms
    llifs_runtime_delta_oversized_total
    llifs_reset_per_sec                  llifs_wake_latency_ms
    llifs_hibernate_delta_bytes          llifs_ram_reclaimed_bytes
    llifs_fork_latency_ms                llifs_inplace_rewind_rejected_total
    llifs_resident_active_ratio          llifs_per_agent_substrate_overhead_bytes
  Restore hazards:
    llifs_restore_hazard_refresh_latency_ms
  Coverage / warmth / drift:
    llifs_startup_pack_coverage_ratio    llifs_resume_pack_coverage_ratio
    llifs_base_content_coverage_ratio    llifs_memory_base_warmth_ratio
    llifs_profile_miss_ratio             llifs_memory_base_overfetch_bytes
  Source / hedging / breakers:
    llifs_blocks_from_ram_total          llifs_blocks_from_ssd_total
    llifs_blocks_from_peer_total         llifs_blocks_from_registry_total
    llifs_p0_hedged_requests_total       llifs_p0_hedge_wins_total
    llifs_origin_breaker_open_total      llifs_origin_hedge_denied_total
    llifs_peer_ejections_total
  Cache / GC / domains:
    llifs_cache_evictions_total          llifs_snapshot_write_backpressure_total
    llifs_registry_fallback_total
```

```
  Copy-up:
    llifs_copyup_full_file_fetches_total llifs_copyup_policy_denied_total
  Revocation (§14):
    llifs_revocation_state_age_seconds   llifs_revoked_object_denied_total
    llifs_revocation_propagation_latency_ms
  Security:
    llifs_corrupt_source_ejections_total

OBS-6: Metric names are recommended; the decomposition they enable (OBS-1) is
       required. The following counters MUST be present (under any names): per-tier
       source attribution (so a start attributes to RAM/SSD/peer/registry),
       origin-hedge denials and origin-breaker openings (§8 ORIGIN-3), and
       corrupt-source ejections (OBS-4).

------------------------------------------------------------
```

13.3 Trace events

```
  image.resolve.{start,done}        rasm.resolve.{start,done}
  materialization.assurance.{start,done}   profile.select.{start,done}
  bootstrap.fetch.{start,done}      pack.fetch.{start,scatter,verify.done}
  mount.ready                       container.exec               container.ready
  block.fault                      mmap.fault                   copyup.detected
  block.fetch.{start,hedge.start,done}      block.verify.done
  origin.breaker.{open,half_open}   hydration.done   source.ejected (security)
  snapshot.freeze.{start,done}      snapshot.write.done          state_root.publish.done
  resume.start                     memory.restore.done          unfreeze.done
  fork.{start,done}                reset.done                   wake.done
  hazards.cleared                  revocation.denied

OBS-7: A start/resume MUST emit enough trace spans to reconstruct the TTFE/TTR
       decomposition (OBS-1) end to end.

------------------------------------------------------------
```

13.4 Primary SLOs

```
  p50/p95/p99 time-to-first-exec        p50/p95/p99 time-to-resume
  p99 foreground fault latency          p99 memory fault latency
  p99 mmap fault latency                p99 time-to-ready
  startup/resume coverage ratio         proof coverage ratio
  base warmth ratio                     critical-path network bytes
  origin-registry bytes per rollout     runtime lazy-materialization failure rate
  reset / wake latency

OBS-8: p99 TTFE, p99 TTR, and p99 foreground/memory fault latency are the load-
       bearing SLOs; the substrate SHOULD optimize them before average full-pull
       or full-hydration time.

------------------------------------------------------------
```

14. Security model

```
The security model is one sentence made enforceable: every byte exposed to a
workload is verified against a trusted Terrapin identifier reachable from a trust
root, and transport is never a trust anchor. This section names the trust
boundary, the requirements that enforce it, revocation, and what is deliberately
out of scope.

------------------------------------------------------------
```

14.1 Trust boundary

```
Trusted: the local kernel and container runtime; imagefsd within the node trust
boundary; configured trust roots; signed base descriptors and their assurance
```

records (§3); the runtime attestation envelope (§2.5); trusted runtime
requirements (§7.2); the revocation policy source (§14.3).

Untrusted or partially trusted: peers; origin-registry transport; mirrors/CDNs;
mutable tags; profiles (performance hints only); the network path; unverified
local cache entries; compressed bytes before verification.

14.2 Requirements

SEC-1: Every byte exposed to a workload (filesystem, memory, or proof) MUST be
verified against a trusted Terrapin identifier (§2.2 TERRAPIN-4, §5
MVERIFY-1). Peers, mirrors, registries, caches, packs, and decompressed
bytes are transport, never trust anchors (§8).
SEC-2: A Terrapin identifier MUST be reachable from a trust root: a base
descriptor validated under the builder regime, or a State Root validated
under the runtime regime (§2.5 DIGEST-BIND-4/6). A trusted State Root does
NOT confer trust on its base; both MUST be validated (SR-2).
SEC-3: Profiles MUST NOT alter verification or identity (PROFILE-1); only trusted
runtime requirements compel (§7.2).
SEC-4: The two binding regimes (builder-attested base, runtime-attested per-agent
state) MUST NOT be substituted (DIGEST-BIND-5).
SEC-5: Restore-hazard hooks (§5.6 RHAZARD-1..7) are security-load-bearing for
multi-tenant restore from a shared base; a base lacking required hooks MUST
NOT be used for multi-tenant restore.
SEC-6: Cache domains MUST be enforced before peer discovery / Have() and for
dedup, keying, warmth, and GC (§9); content-presence side channels MUST be
contained within a domain. Cross-domain sharing is opt-in (§9 SHARE-1).
SEC-7: Peer/source corruption MUST be observable and actionable: a block failing
verification MUST eject/penalize the source and MUST emit a security event
(§13 OBS-4); repeated verification failure for one object SHOULD trigger
alerting.
SEC-8: Regardless of isolation, unverified decompressed bytes MUST NEVER reach VFS
or guest memory (§8 COMP-3); decompression and untrusted-metadata parsing
SHOULD additionally run sandboxed (§8 COMP-4).
SEC-9: The asserted assurance mode (§3) MUST be reported as a weaker mode and MUST
NOT claim node-verified OCI↔rootfs equivalence.
SEC-10: A mutable tag MUST NOT be treated as a trust root: it MUST be resolved to
an immutable OCI image digest and a validated base descriptor before any
byte is exposed (§8 REG-4, §7 PROFILE-3). A gap between tag discovery and
digest binding MUST NOT widen trust.

14.3 Revocation

REV-1: Signatures and attestations (base descriptors, materialization
attestations, runtime attestations) SHOULD have short validity windows
unless backed by an online revocation check.
REV-2: The substrate SHOULD support revocation sources (transparency-log inclusion
policy, CRL/OCSP-like checks, or signed deny lists); revocation state MUST
be cached with an expiry.
REV-3: If a base descriptor (or its committed OCI image digest, §2.5
DIGEST-BIND-3), profile record, runtime-requirements policy, or State Root is
revoked, the control plane MUST stop new scheduling/resume against it.
Revocation evaluation MUST include both lineage enumeration (§6.3
LINEAGE-2) and State Root `base`-reference resolution: revoking a base MUST
deny every retained State Root whose `base` reference (§6.2) resolves to
that base descriptor, and thus its OCI image digest. (State Roots do not themselves carry
an OCI subject; the chain is State Root.base → base descriptor → OCI digest.)
REV-4: Policy MUST define whether already-running agents bound to a revoked base
are allowed to continue, drained, killed, or isolated.
REV-5: Revocation propagation latency SHOULD be observable (§13).

14.4 Deliberately out of scope (deployment must supply)

This document specifies verified state delivery; it does NOT by itself provide workload authentication/authorization or network egress policy. Until those are added, a deployment relying on LLIFS alone for isolation MUST run trusted workloads in an isolated cluster.

SCOPE-1: Workload authn/authz and network/egress policy (e.g. NetworkPolicy) are NOT provided by LLIFS and MUST be supplied by the surrounding platform for untrusted multi-tenant workloads.
SCOPE-2: High-availability/multi-replica of the control plane, and full control-plane sharding, are out of scope here; the cache-domain keying and content-addressed identity (§9) and the byte-exact encodings (§15) are fixed so they can be added without changing identity.
SCOPE-3: At-rest encryption of private memory/filesystem/runtime deltas (§6) SHOULD be applied in the cold/blob tier for confidential deployments; encryption disables cross-tenant dedup of those objects (which §9 already forbids by default), so it composes with the per-domain keying without weakening it.

---

15. Canonical encodings and conformance vectors

Every identity in LLIFS is a Terrapin digest over a byte-exact canonical encoding, so independent implementations agree or reject (the supply-chain equivalence claim, MAT-20). This section is the registry of those encodings, pointing to the two already defined inline, defining the remaining (agent-state) ones, and pinning the conformance vectors that make them testable.

15.1 Shared primitives

All canonical encodings in this document use the §3.2 primitives (PRIM-1..5): u8/u16/u32/u64 unsigned little-endian; i64 two's-complement little-endian; varbytes = u32 length prefix + raw bytes; a 32-byte digest field is the raw Terrapin identifier (the G(manifest) payload, never the hex string, never the bare tree root). Unless stated otherwise, every canonical document is length-framed and a parser MUST reject trailing bytes.

15.2 Encoding registry

filesystem logical tree	LLT1	§3.2	→ logicalRootfsDigest
filesystem physical layout	LLAY1	§4.3	→ layoutDigest
State Root	LLSR1	§15.4	→ State Root identifier
base descriptor	LLBD1	§15.5	→ base descriptor identifier
memory layout	LLML1	§15.5	→ memory layout identifier
memory delta (page-indexed)	LLMD1	§15.6	→ memory delta identifier
filesystem (overlay) delta	LLFD1	§15.7	→ filesystem delta identifier
runtime delta (raw opaque)	LLRD1	§15.4	→ runtime delta identifier
workload identity	LLWI1	§15.8	→ workload_identity
sandbox pin	LLSP1	§15.9	→ sandbox pin (external sha256)

ENC-1: A new object's identity MUST be the Terrapin identifier of its canonical encoding here (the sandbox pin is the exception: an external-plane sha256, §15.9). LLT1/LLAY1 (§3.2/§4.3) are normative as written and not restated.
ENC-2: Every canonical document MUST begin with its 4-byte magic and a u16 version, MUST length-frame its records, and MUST reject trailing bytes and unknown enum values (raw opaque runtime-native objects are the exception: ENC-3).
ENC-3: Opaque runtime-native objects, the base runtime_state blob (§15.5) and the runtime delta (LLRD1, §15.4), are RAW Terrapin byte objects: LLIFS defines no magic, version, or framing and does not parse them; their identifier is terrapin-sha256 over the exact runtime-produced bytes, and compatibility is gated externally by the sandbox pin and compatibility matrix (§11.4, MATRIX-3).

```
       Every other encoding in this registry is an LLIFS-framed structural object
       (ENC-2).

------------------------------------------------------------
```

15.3 Terrapin profile conformance vectors

```
CROSS-CONFIRMED (ENC-CONF-2 satisfied): every vector below was reproduced
byte-for-byte by two independent v0.3 implementations, terrapin-rs (Rust,
streaming/parallel) and terrapin-go (Go, in-memory plus a streaming reader for the
128 GiB cases), and both agree with these values. The two have independent canonical
manifest encoders, so this also rules out silent manifest divergence.

  Constants:
    G(empty)                = 473a0f4c3be8a93681a267e3b1e9a7dcda1185436fe141f7749120a303721813
    G(2 MiB zero block)     = 67cbed9b97ddabde2863f4daefa4f57176567a7c3ccfa1560c1065f9c8af74d6

  Identifier vectors (terrapin-sha256:<hex> = G(canonical manifest)):
    empty (len 0)           = f4b8abc1cfd6ffec75b4070be5440706286b3a7af937ef5d020ca2c0c1210458
    "hello world" (len 11)  = 7bc0163f32e5f6082308ae0dff3dc7c9b0488e5aa652d9de01418df5ec800c8c
    one zero byte (len 1)   = dce39f984d9c140e4ad8f4b448a2ae6ae5398ed1adbb4d07ed8bedbc5b3b4598
    block-1 (2097151 zero)  = dc7f0a33cf02e7a84fc380a41d396b451c96325a633a87528ebf797621befad7
    one block (2097152 zero)= 6fbd6447c2d8d70a83ae159461847a1a410679900702433dd2b04d063a3b2f9b
    block+1 (2097153 zero)  = 5ba8049ae8f68a47acd4fad265c8a963aa82735e90f209dd79ff8d6d2188fdc5
    128 GiB zero (65536 blk)= 8d03319328c6d6b3cd00566d894443b2a82d31437b580ee533c2021d82bdb5a4
    128 GiB + 1 byte        = 6f552f944f4995878c7facc92c29c3643aaafc2a5bff90e255bbf430210d551b

  Per-level all-zero subtree roots (SPARSE-8; Z_0 = G(2 MiB zero block); Z_L =
  G(Z_{L-1} repeated 65536 times), i.e. one full 2 MiB hash-file block of the lower
  level, fanout = 2097152/32 = 65536):
    Z_0 (leaf, <= 2 MiB)        = 67cbed9b97ddabde2863f4daefa4f57176567a7c3ccfa1560c1065f9c8af74d6
    Z_1 (<= 128 GiB)            = 9e7e7e12b71c2b008302a4e4f5abe5b012025a8bd59d9ea5aa187f187a165599
    Z_2 (<= 8 PiB)              = a9b835a482cdf112d81ee77942204e5a96895fc14907bd6f049b66d3e43e42eb
    Z_3 (<= 512 EiB, full u64)  = 96df0f67df694e2f6cbbfb1b978415dc51b3000956d4e09bcd5dea1031129e35
  Z_0 and Z_1 are vector-validated: Z_0 equals the G(2 MiB zero block) constant, and
  Z_1 is the tree root of the 128 GiB zero vector (G(manifest(137438953472, Z_1))
  equals the 128 GiB identifier above). Z_2 and Z_3 follow from the same node-hash
  recurrence over the agreed G; both oracles compute them identically.

Manifest accept/reject (canonical Terrapin manifest, §2.2): a manifest MUST be
ASCII, LF-terminated (including the last line), field order exactly terrapin,
block_size, length, tree, exactly one space after each colon, integers with no
leading zeros, tree exactly 64 lowercase hex. Each of these MUST be rejected (not
normalized): uppercase hex; missing final LF; wrong field order; leading zeros;
extra spaces; unknown/extra keys, comments, or blank lines; block_size != 2097152;
the bare tree root presented as the identifier.

------------------------------------------------------------
```

15.4 State Root encoding (LLSR1)

```
  magic "LSR1"; version u16=1; then fields in fixed order; varbytes fields are
  length-framed (so "none" is an explicit length-0 value and optional digests are
  length 0 or 32, never a mix), and the createdSec/createdNsec fields use their
  stated fixed widths:
    actor (varbytes); parent (0 or 32 = State Root id); base (32 = base descriptor
    id); memory (0 or 32 = memory delta id); filesystem (0 or 32 = filesystem delta
    id); runtime (0 or 32 = runtime delta id); sandbox_pin (0 or 32 = sha256);
    workload (0 or 32 = workload_identity);
    run_mode (varbytes of one byte: 1=clean 2=golden 3=persist-rootfs
    4=persist-rootfs+memory); producer (varbytes); createdSec (8-byte i64);
    createdNsec (4-byte u32, 0 <= nsec < 1e9).
  State Root identifier = Terrapin identifier of these bytes.

ENC-SR-1: All fields MUST be present in this order; "none" is the explicit length-0
          value, never omitted (OBJ-5, SR-1). A digest field's length MUST be
          exactly 0 or 32; any other length MUST reject. Unknown run_mode MUST
          reject. createdNsec >= 1e9 MUST reject.
ENC-SR-2: Exactly the §6.2/SR-4 conditional rules apply: if memory or filesystem
```

```
          is non-none, workload MUST be non-none; if memory is non-none, sandbox_pin
          MUST be non-none. When memory is non-none, runtime is normally non-none
          and is "none" only under the RDELTA-1 no-runtime-state policy exception;
          restore enforces RDELTA-5.

Runtime delta (LLRD1), the State Root's runtime field (§6.1 DELTA-2R):

  The runtime delta is a RAW opaque runtime-native Terrapin object: LLIFS defines no
  magic, version, or framing and does not parse it, exactly like the base
  runtime_state blob (§15.5 ENC-BD-3). Its identifier is terrapin-sha256 over the
  exact runtime-produced bytes (ENC-3).

ENC-RD-1: The runtime delta MUST contain ONLY runtime-native execution metadata
          needed to reapply the diverged non-memory, non-filesystem state. Guest
          memory page contents MUST be carried in the memory delta (LLMD1) and
          writable-upper contents in the filesystem delta (LLFD1); the runtime delta
          MUST NOT be a backdoor "opaque checkpoint" for memory or filesystem bytes.
ENC-RD-2: A restorer MUST interpret a runtime delta ONLY under a runtime satisfying
          the State Root's sandbox pin (§2.5) and the base descriptor's compatibility
          matrix (§11.4, MATCH-COMPAT MATRIX-3), and MUST apply it only after the base
          runtime-state blob (RDELTA-5); an incompatible or missing-required runtime
          delta MUST fail closed (RDELTA-1/5). A future STRUCTURED runtime delta would
          be a separate LLIFS-framed object with its own magic, not this opaque form.

------------------------------------------------------------
```

15.5 Base descriptor encoding (LLBD1)

```
  magic "LBD1"; version u16=1; then fields in this exact order; varbytes fields
  length-framed, fixed-width fields (32-byte digests, u8, u32) at the stated width:
    oci_image_digest (varbytes: the ASCII string "sha256:<64 lowercase hex>",
    ENC-BD-4); logical_rootfs (32); layout (32);
    base_memory (32); runtime_state (32); memory_layout (32); sandbox_pin (32
    sha256); compat_matrix (the CompatMatrix sub-record below); assurance_mode
    (u8: 1=asserted 2=derived-attested 3=node-derived); runtime_state_policy (u8:
    0=runtime-state-required, the default; 1=no-non-memory-runtime-state-required-
    for-memory-resume, the RDELTA-1 exception, ENC-BD-5).

  CompatMatrix (fixed field order, each as typed): runtime_id (varbytes);
    runtime_version (varbytes); platform_mode (u8: 1=ptrace 2=systrap 3=KVM);
    arch (varbytes, GOARCH); page_size u32; cpu_feature_mask (varbytes, opaque
    fixed-width feature bitset bytes); checkpoint_format_version u32; abi_tag
    (varbytes); seccomp_tag (varbytes); cgroup_tag (varbytes); namespace_tag
    (varbytes).
  Base descriptor identifier = Terrapin identifier of these bytes (the signed
  subject, §2.5 DIGEST-BIND-3).

ENC-BD-1: All fields and CompatMatrix sub-fields MUST appear in this exact order
          and width; unknown platform_mode or assurance_mode MUST reject; trailing
          bytes MUST reject. No tag is sorted (each is a single value, not a list);
          all are length-framed so empty is length-0, distinct and intentional.
          cpu_feature_mask is OPAQUE producer-captured bytes committed verbatim:
          its width and bit assignment are arch-defined and are NOT independently
          re-derived for byte-identity (the descriptor commits exactly the bytes the
          producer captured). A restorer compares its own capabilities against the
          mask using the arch's interpretation (§11.4 MATRIX), but the committed
          bytes are authoritative for identity.
ENC-BD-2: The base descriptor is the signed trust subject; a verifier MUST reject a
          descriptor that does not parse to exactly this canonical form.
ENC-BD-3: runtime_state is a runtime-native OPAQUE byte blob: LLIFS defines no
          internal structure for it, and its Terrapin identifier covers its exact
          stored bytes verbatim. (It is runtime-version-specific; compatibility is
          gated by the compatibility matrix, §11.4.)
ENC-BD-4: oci_image_digest MUST be the canonical OCI digest STRING form, the exact
          ASCII bytes "sha256:" followed by exactly 64 lowercase hex characters, and
          nothing else (it is the external-plane identity, §2.1; raw 32-byte form,
          uppercase hex, other algorithms, or a missing prefix MUST reject). It is
          the only base-descriptor field in the external plane; all other digest
          fields are raw 32-byte Terrapin identifiers.
ENC-BD-5: runtime_state_policy is the canonical signed NO-RUNTIME-STATE ASSERTION
          (RDELTA-1): value 1 asserts that NO non-memory runtime state is required to
```

```
          resume ANY memory-persisting State Root on this base under this policy,
          because the runtime can reconstruct all required execution metadata from the
          committed memory and filesystem state plus signed runtime policy. It is NOT
          the weaker claim that the base's restore point merely happened to hold none
          (a running agent can later accrue fd offsets, epoll/timer/signal/
          syscall-restart/socket state); it is the resume-time guarantee for later
          snapshots. It permits a memory-persisting State Root to set runtime "none".
          Because it is committed by the signed base descriptor (ENC-BD-2),
          restore validates it as part of base-descriptor validation (DIGEST-BIND-3/
          6); a State Root with memory non-none and runtime "none" MUST reject unless
          its base descriptor carries runtime_state_policy==1. Unknown values MUST
          reject.

Memory layout encoding (LLML1), the base descriptor's memory_layout field (§5.3):

  magic "LML1"; version u16=1; page_size u32; regionCount u64; regions (sorted by
  guestStart, non-overlapping), each: guestStart u64; length u64; state u8
  (1=mapped-data 2=mapped-zero 3=unmapped/no-access, §5.3 MLAYOUT-5); objectOffset
  u64 (offset into the base memory snapshot object for mapped-data; 0 otherwise);
  prot u8 (bit0 R, bit1 W, bit2 X); flags u8 (bit0 guard-page, bit1 shared else
  private).
  Memory layout identifier = Terrapin identifier of these bytes.

ENC-ML-1: regions MUST be sorted by guestStart, non-overlapping, and tile the
          committed guest address space [0, committedEnd) exactly (committedEnd =
          guestStart+length of the last region); unknown state/prot/flags bits MUST
          reject; trailing bytes MUST reject. This is the byte-exact form of the
          §5.3 MLAYOUT-1..5 mapping (linear order, three-state mapping, page size,
          protections); it commits the per-range mapping state and attributes that
          restore reproduces.
ENC-ML-2: The base memory snapshot is guest-address-linear (MLAYOUT-1): for a
          mapped-data region (state 1) objectOffset MUST equal guestStart; for
          mapped-zero (2) and unmapped/no-access (3) objectOffset MUST be 0 and the
          region occupies no stored bytes, contributing canonical zeros to the
          base-memory identity (MEM-1). The base memory snapshot object's manifest
          length MUST equal committedEnd. Because the MEM-1 identity does not
          distinguish states 2 and 3 (both zeros), this signed layout is the sole
          authority for that distinction (MEM-5); any other objectOffset MUST
          reject.

------------------------------------------------------------
```

15.6 Memory delta encoding (LLMD1)

```
The memory delta is a page MAP: for each diverged guest page, where its bytes come
from (§6.1). The map directly names every page's source, so resume needs no
ancestor traversal (§6.3 LINEAGE-3) and a fork shares parent/base pages by direct
reference with no copy.

  magic "LMD1"; version u16=1; page_size u32; base_ref (32, the base memory
  snapshot object); entryCount u64; entries, each:
    guestPageIndex u64;
    source u8 (1=this-delta-data 2=ancestor-delta-data 3=base 4=canonical-zero);
    sourceObjectId 32 (the data object for source 1/2; all-zero for source 3/4);
    offset u64; length u64
  sorted ascending by guestPageIndex, unique. The delta's own changed pages live in
  a separate "delta-data" Terrapin object referenced by source==1 entries.
  Memory delta identifier = Terrapin identifier of these (map) bytes.

ENC-MD-1: The encoding MUST be canonical (one form per restored memory state).
          entries MUST be sorted and unique by guestPageIndex. A page identical to
          the base MUST be OMITTED (it resolves from the base copy-on-write, §5.4);
          source==3 (base) is used ONLY by a fork to explicitly revert a page the
          parent had changed back to the base value, never to redundantly list a
          base-identical page. For a fork, unchanged-from-ancestor pages use
          source==2 naming, by absolute Terrapin id, the ANCESTOR delta-data object
          that actually holds the bytes (resolved ONCE at fork time by reading the
          parent's map, the §6.3 FORK-3 flatten), so the child map is self-contained
          and resume never consults any ancestor map; changed pages use source==1.
          Unknown source MUST reject; trailing bytes MUST reject.
ENC-MD-2: Each entry covers exactly one guest page: length MUST equal page_size.
```

```
          For source==1/2, [offset, offset+length) MUST be in-bounds of the named
          data object; source==2's sourceObjectId MUST be the absolute Terrapin id of
          the ancestor delta-data object holding the bytes, which resume reads
          directly and MUST NOT resolve through any ancestor's page map (LINEAGE-3).
          For source==3 (base), sourceObjectId MUST be all-zero and
          offset MUST be the page's guest offset within the base memory snapshot.
          For source==4 (canonical zero), sourceObjectId MUST be all-zero and
          offset MUST be 0. On restore each page is resolved from its named source
          and verified (§5 MVERIFY) before exposure (source==4 installs a verified
          zero page, §5.4 COW-6b); every source object is a separate Terrapin
          object.
ENC-MD-3: The delta's own changed pages MUST form exactly ONE canonical
          "this-delta-data" Terrapin object: the source==1 pages concatenated in
          ascending guestPageIndex order. For each source==1 entry, sourceObjectId
          MUST equal that one object's Terrapin id, length MUST equal page_size, and
          offset MUST equal (the page's ordinal among source==1 entries) * page_size.
          This makes LLMD1 canonical: the same changed-page state over the same
          base/parent yields one memory delta identifier.
ENC-MD-4: A memory delta MUST be directly resolvable (DELTA-5): every entry names,
          by absolute Terrapin id, the exact object holding its bytes (this-delta-
          data, an ancestor delta-data object, or the base via base_ref), and resume
          reads those named objects directly, never walking lineage or consulting any
          parent delta's map. base_ref and the source==2 ancestor object id are
          DIRECT resolution inputs; only the parent State Root link and lineage chain
          (§6.3) are provenance, GC, and compaction inputs.
ENC-MD-5: Source selection MUST be canonical (one encoding per effective page state),
          in this precedence: (a) OMIT a page byte-identical to the base (it resolves
          from base copy-on-write, §5.4); (b) use source==3 (base) ONLY to revert a
          fork page the parent had changed back to the base value, never for an
          already-base-identical page (omitted per (a)); (c) use source==4 (canonical
          zero) ONLY when the page is all-zero AND the base page at that offset is not
          all-zero (an all-zero page equal to the base is omitted per (a)); (d)
          otherwise use source==1 for a changed page and source==2 for a page carried
          unchanged from an ancestor. A verifier MUST reject structurally detectable
          violations (ENC-MD-2) and, where it has the base, a redundant source==3/4
          for a base-identical page.

------------------------------------------------------------
```

15.7 Filesystem (overlay) delta encoding (LLFD1)

```
The canonical overlay-delta object reconstructing the writable upper, §6.1. It is
a FINAL-STATE delta: it encodes the resulting writable upper directly, not a replay
log, so there are no multi-step operations (a rename is encoded as its result: a
put at the new path and a whiteout at the old). It is SELF-CONTAINED against the
base lower: a resume reconstructs the upper from this object alone, plus the content
objects it names by absolute id and the base rootfs the State Root references, never
by walking parent deltas (DELTA-5, LINEAGE-3). Op tails reuse the LLT1 common prefix
exactly.

  magic "LFD1"; version u16=1; parent_ref (varbytes: 32-byte filesystem delta id, or
  length-0 = none; PROVENANCE/GC/compaction only, never required to reconstruct the
  upper, DELTA-5); opCount u64; ops (sorted by normalized path bytes), each an
  OpRecord:
    path (varbytes, normalized per §3.2);
    op u8 (1=put-file 2=put-dir 3=put-symlink 4=put-chardev 5=put-blockdev
           6=put-fifo 7=whiteout);
    then the op tail:
      put-* common prefix (exactly LLT1 §3.2 widths, in order): mode u32; uid u32;
        gid u32; mtimeSec i64; mtimeNsec u32; xattrCount u32; xattrs (XattrRecords
        sorted by key, unique; §3.2);
      then the type tail: put-file → contentId 32 + contentLen u64; put-symlink →
        target varbytes; put-chardev/put-blockdev → major u32 + minor u32;
        put-dir/put-fifo → (none);
      whiteout → (no tail).

ENC-FD-1: ops MUST be sorted by normalized path and paths MUST be unique (exactly
          one op per path; a path cannot be both put and whiteout), and all paths
          MUST normalize per §3.2 (reject absolute/NUL/".."). Because the delta is
          final-state, there is no rename op and no multi-step conflict to resolve.
          hardlinkGroup is NOT represented: an overlay copies up whole files, so a
```

```
          put-file carries its own contentId (a later compaction MAY dedupe by
          content object, but the delta encodes no hardlink group). Unknown op MUST
          reject; trailing bytes MUST reject.
ENC-FD-2: whiteouts MUST be representable so deletions survive restore. The
          filesystem delta identifier is the Terrapin identifier of these bytes;
          content objects are separate.
ENC-FD-3: The filesystem delta MUST be directly resolvable (DELTA-5): it encodes the
          COMPLETE final-state writable upper and every content reference is an
          absolute content-object id, so resume reconstructs the upper from this
          object, its content objects, and the base lower alone. parent_ref is the
          lineage edge for provenance/GC/compaction and MUST NOT be consulted on the
          resume critical path; a compaction MAY rewrite a parent-relative storage
          form into this complete form, but resume MUST NOT depend on traversal.

------------------------------------------------------------
```

15.8 Workload identity encoding (LLWI1)

```
The §6.2 SR-5 canonical workload identity (an internal Terrapin identity, not an
external digest).

  magic "LWI1"; version u16=2; then a pod block, then containers.

  Pod block: pause_image (varbytes); hostname (varbytes); host_flags u8 (bit0
    hostNetwork, bit1 hostPID, bit2 hostIPC, bit3 shareProcessNamespace); pod
    security (SecurityContext sub-record below).

  containerCount u64; containers (in declared order), each:
    name (varbytes); image (varbytes);
    cmdCount u64 + each (varbytes, declared order);
    argsCount u64 + each (varbytes, declared order);
    envCount u64 + each (name varbytes; value varbytes), SORTED by name then value;
    working_dir (varbytes);
    security (SecurityContext sub-record);
    capAddCount u64 + caps (varbytes, SORTED, unique);
    capDropCount u64 + caps (varbytes, SORTED, unique);
    mountCount u64 + mounts (SORTED by mountPath), each: mountPath (varbytes);
      subPath (varbytes); sourceName (varbytes); sourceType (varbytes); mflags u8
      (bit0 readOnly);
    deviceCount u64 + devices (SORTED by path), each: path (varbytes);
    mem_limit u64 (bytes, 0 = unset); cpu_limit_milli u64 (0 = unset).

  SecurityContext sub-record: runAsUser i64 (-1 = unset); runAsGroup i64
    (-1 = unset); sflags u8 (bit0 privileged, bit1 readOnlyRootFilesystem, bit2
    allowPrivilegeEscalation, bit3 runAsNonRoot); seccomp (varbytes); apparmor
    (varbytes); selinux (varbytes).

  workload_identity = Terrapin identifier of these bytes.

ENC-WI-1: Fields are the restore-visible projection of atelet.WorkloadSpec (which
          carries them); ateom's reduced Container is insufficient (§12). Every list
          is ALWAYS present with its count; absent and empty lists are identical
          (count 0). env, capAdd, capDrop, mounts, and devices MUST be sorted as
          stated and unique by their sort key (a duplicate env name, capability,
          mount path, or device path MUST reject). Unset scalars use their stated
          sentinel (i64 -1, u64 0, varbytes length 0). The encoding MUST be
          deterministic. (This is the value of the State Root `workload` field, §6.2;
          "workload_spec_digest" is a deprecated alias for workload_identity.) The
          exact field set MUST be reconciled against atelet.WorkloadSpec before §15
          is frozen. This is a HARD freeze blocker, not a nicety: a workload-identity
          mismatch is a restore-safety bug, so any WorkloadSpec field that affects
          restore correctness MUST be added here (SR-5) before v1.0 freezes.
ENC-WI-2: Fields intentionally EXCLUDED, because they do not affect restored state or
          are regenerated post-restore, MUST NOT be committed: scheduling and
          placement (node name, affinity, tolerations, priority, topology); status
          and conditions; liveness/readiness/startup probes; restart policy; image
          pull policy and pull secrets (provenance, not restore state);
          purely-informational labels and annotations; dynamically assigned values
          (pod IP, assigned ports, service-account token contents) and other
          RHAZARD-refreshed state (§5.6); and volume CONTENTS (the data lives in the
          filesystem plane; only mount TOPOLOGY is committed, above). An annotation
```

```
          that materially changes runtime execution is NOT informational and MUST be
          committed (SR-5).

------------------------------------------------------------
```

15.9 Sandbox pin encoding (LLSP1)

```
Per-architecture, external-plane identity of the runtime assets (§2.5).

  magic "LSP1"; version u16=1; sandbox_class (varbytes); goarch (varbytes);
  assetCount u64; assets (SORTED by name bytes), each: name (varbytes),
  asset_sha256 (32). AssetFile.url is excluded (transport hint, not identity).
  sandbox pin = sha256 over these bytes (external plane, §2.1).

ENC-SP-1: The pin covers exactly one GOARCH; other architectures' assets are not
          included (§2.5 DIGEST-BIND-7). sandbox_class and goarch MUST be non-empty;
          the asset list MUST be non-empty; assets MUST be sorted by name and
          unique; trailing bytes MUST reject.

------------------------------------------------------------
```

15.10 Conformance and the reference oracle

```
ENC-CONF-1: A conforming implementation MUST reproduce the §15.3 vectors and the
            LLT1/LLAY1 golden vectors, and MUST agree byte-for-byte on every
            canonical encoding here or reject identically.
ENC-CONF-2: A reference oracle (a non-production tool sharing the Terrapin core)
            SHOULD emit the vector set as the cross-implementation conformance
            target, covering: the §15.3 Terrapin vectors; LLT1 cases (sorted
            entries, xattrs, hardlink pair, sparse vs dense same identity,
            zero-length file, and the reject cases: absolute/NUL/".." paths,
            duplicate path, bad type, duplicate xattr, mtimeNsec overflow, count
            mismatch, trailing bytes); LLAY1 cases (tiling extents, virtual-zero,
            empty and hardlink files, plus the reject cases including backing==2 /
            non-zero inlineCount, which are reserved in v1, LLAY1-11); a memory-layout
            (LLML1) case with a mapped-data region at a HIGH guest address preceded by
            a very large unmapped/no-access zero span (to catch tree-height and
            off-by-one bugs in the virtual zero-subtree proof rules, SPARSE-8), plus a
            mapped-zero vs unmapped state-mismatch reject case; the per-level all-zero
            subtree roots Z_0..Z_L the 2 MiB profile uses (SPARSE-8 mandates pinning
            these so virtual proof blocks are byte-identical across implementations);
            an LLSR1 round trip WITH a non-none runtime delta present, and an LLBD1
            round trip with runtime_state_policy==1 plus a memory-non-none /
            runtime-none State Root that MUST reject without it (RDELTA-1, ENC-BD-5);
            and at least one round trip for LLSR1/LLBD1/LLMD1/LLFD1/LLWI1/LLSP1 plus
            their reject cases. The runtime delta and base runtime_state blob are raw
            opaque objects (ENC-3): they have NO parse reject cases; only an identity
            round trip (terrapin-sha256 over fixed bytes) is exercised.
ENC-CONF-3: The production digest-producing code SHOULD be a single implementation
            (the digest boundary is the language boundary) so independent
            implementations cannot silently diverge; the oracle's vectors are the
            agreement test.

------------------------------------------------------------
```

16. Phase plan and minimal viable surface

```
The core bet: injecting exactly the right verified bytes at startup/resume beats
making the pipe wider, and density comes from ONE verified base shared
copy-on-write across many sandboxes. The first gate is therefore a single
capability, not the whole lifecycle surface: shared-copy-on-write base memory in
gVisor (§11.2 GVISOR-3). Phase 1 proves exactly that gate on one node (nothing
more); Phase 2 builds the persisted lifecycle and production latency on top;
Phase 3 adds yield, fan-out, and fleet behavior; Phase 4 optimizes with native
backends. Each phase has a success criterion.

------------------------------------------------------------
```

16.1 Phase 1: prove the density gate (one node)

```
Phase 1 is NOT the full lifecycle product; it is the shortest path to proving the
density mechanism, built as a dependency ladder:

  Rung 1 - Verified lazy filesystem. Terrapin verification (§2.2) + LLT1/LLAY1
    (§3, §4) served through a CAS-backed gofer (gVisor) or FUSE lower with
    verify-before-expose lazy reads, singleflight, P0 priority, hedged fetch +
    origin breaker, registry/mirror fallback (§8); OverlayFS upper + copy-up (§4.5).
    A conformance oracle emits the §15 vectors (§15.10). Gate: a multi-GiB image
    execs without full hydration; every exposed byte verifies.
  Rung 2 - Lazy restore page delivery. Reuse gVisor background restore (§11,
    GVISOR-2) with the page source redirected into the CAS-backed gofer/page
    provider: a base memory snapshot (§5.1, §5.3) faults lazily, verify-before-
    expose, with the absent / known-zero / unmapped hazard handling (§5.4
    COW-6/6a/6b) and the restore working set (§5.5). Pages are still materialized
    per sandbox here; density arrives at rung 3. Gate: a sandbox restores and runs
    faulting only what it touches, and an absent range never reads as zero.
  Rung 3 - Shared copy-on-write base (THE DENSITY GATE). The one net-new runtime
    capability (§11.2 GVISOR-3): N sandboxes MAP_PRIVATE one verified base file so
    resident base pages are physically shared and only written pages go private.
    Until this passes, everything above is a latency optimization, not a density
    breakthrough. Gate: the acceptance test below passes.

  Do NOT build P2P, fork, yield, or the full delta/State-Root lifecycle in Phase 1.
  The control-plane binding (§12) lands in Phase 2 with hibernate/resume.

Acceptance test (the density gate, deliberately brutal). Restore N sandboxes from
one base and prove:
  - a base block is FETCHED once and VERIFIED once, independent of N (verification
    cost does not scale with agent or fault count, MVERIFY-2);
  - the same host PHYSICAL pages back all N sandboxes (one resident copy per cache
    domain, COW-1/COW-3);
  - a write creates a private page for the writer ONLY (copy-on-write, no bleed);
  - an absent (not-yet-fetched) range NEVER reads as zero: it traps, fetches, and
    verifies before exposure (COW-6/6a);
  - a signed known-zero range installs a verified zero page with NO fetch (COW-6b),
    distinguished from absent by the signed layout (MEM-5), never by a memfd hole.
If this passes, the density thesis is real; if not, the rest is a lazy checkpoint
reader, not a density substrate.

Measurement harness (the first prototype, before any platform features). Not a
control plane: a harness around the shared base. Launch a multi-GiB warmed gVisor
workload, capture a base, restore many sandboxes, and report base resident bytes,
proportional set size (PSS) per sandbox, dirty pages per sandbox, Terrapin
verification count, foreground memory-fault p50/p95/p99, absent-range fault
behavior, known-zero behavior, and TTR (§7.6). These numbers, not a feature list,
are what distinguish a real density substrate from a merely lazy restore.

Implementation surface (keep Phase 1 ruthlessly focused on four interfaces; defer
fork, yield, P2P, scheduler warmth, fleet GC, and native backends until physical
sharing and absent-vs-zero correctness are proven under load):
  - BlockRef resolver: object / level / blockIndex -> verified bytes or fail closed.
  - CAS-backed rootfs gofer: path + offset -> content-object BlockRef -> verified
    bytes (GVISOR-1).
  - CAS-backed restore page provider: pages-file offset / guest address -> memory
    layout mapping -> base BlockRef (GVISOR-2, GVISOR-6).
  - Shared MemoryFile backing: N sandboxes MAP_PRIVATE the same verified base pages
    (GVISOR-3).

------------------------------------------------------------
```

16.2 Phase 2: persisted lifecycle and production latency

```
Build the persisted lifecycle on the proven base, then harden latency.

  Lifecycle: overlay→delta capture (§6.1: memory, filesystem, and runtime deltas)
    with self-contained deltas (DELTA-5); the
    write/commit path under the single freeze barrier (§6.4); State Root mint +
    hibernate/resume (§6.2, §6.5) with local/external tiers and run modes (§6.6);
```

```
    then copy-on-write FORK (§6.3) with the authorization hook (FORK-5). Control
    plane: the ateapi SnapshotInfo → StateRootSnapshotInfo migration,
    SnapshotPlacement, CreateActorRequest.from_state_root, and the
    Checkpoint/Restore State-Root binding (§12) are REQUIRED here.
  Latency hardening: RAM CAS; an eBPF/VFS startup+resume profiler and dynamic
    profile refinement; scheduler warmth scoring + resume affinity (§10);
    controller-driven prewarm; registry metadata cache; derived-attested as the
    default assurance mode (§3).

  Success: a hibernated agent resumes from its delta + shared base with no full
  hydration and every byte verified; a fork fans one snapshot into N children; and
  p99 foreground-fault and memory-fault latency stay bounded across realistic
  rollouts while profile drift is detected and re-profiled.

------------------------------------------------------------
```

16.3 Phase 3: yield, fan-out, and fleet

```
Add yield and scale out.

  Yield: hibernate-under-upstream-call plus the durable upstream transaction (§6.5
    YIELD-1..3) with at-most-once submission and the sent_unknown state machine
    (YIELD-2/2a).
  Fleet: the peer protocol (§8.6); rack/AZ-aware source selection and seed-node
    rollout planning; peer health scoring; cross-domain peer policy; cluster-wide
    origin-hedge budget coordination and origin breakers at scale; fork fan-out at
    scale (§10.4). Do NOT build P2P before this phase.

  Success: a blocked agent yields its RAM and wakes to deliver the held response
  with no duplicate non-idempotent upstream execution beyond declared policy; and
  origin-registry bytes per 1,000-node rollout drop substantially while TTFE/TTR
  stay within SLO, a fork of one agent into N children fanning out from peers, not
  origin.

------------------------------------------------------------
```

16.4 Phase 4: native performance path

```
Evaluate native, page-cache-integrated backends (optimized FUSE, EROFS/fscache,
composefs-style verified trees, virtiofs) per §11.5, and the in-place-rewind reset
optimization (§6.5 RESET-4) only if measured to beat restore-fresh.

  Success: a native backend improves p99 fault latency, CPU, and memory footprint
  without weakening logicalRootfsDigest, layoutDigest, Terrapin, State Root, or
  cache-domain guarantees.

------------------------------------------------------------
```

16.5 Minimal viable surface

```
The minimal viable surface is the complete v1 component set, in dependency order
below. Phase 1 (§16.1) builds only the density-gate subset: items (1)-(4) plus the
shared-CoW base of (10); the persisted lifecycle of (5) lands in Phase 2 and the
fleet items (8)-(9) and yield in Phases 2-3. (1) the two digest
planes + Terrapin verification (§2); (2) materialization + LLT1 (§3); (3)
content-addressed layout + LLAY1 (§4); (4) the base memory snapshot + CoW restore
+ RHAZARD (§5); (5) overlay/delta + State Root + lineage + fork + reset/hibernate/
yield + tiers/run-modes (§6); (6) profiles/requirements/packs/bootstrap/coverage +
TTFE/TTR (§7); (7) the verify-before-expose fetch path + singleflight + hedging +
origin breaker + registry fallback (§8); (8) cache domains + keying + leases +
mark-sweep GC (§9); (9) warmth + resume affinity (§10); (10) the gVisor adapter
incl. shared-CoW base (§11); (11) the ateapi/atelet/ateom binding incl. the
proposed proto additions (§12); (12) the canonical encodings + conformance oracle
(§15); (13) baseline observability (§13) and the security/revocation model (§14).
P2P (§8.6, Phase 3) and native kernel backends (§11.5, Phase 4) come later.

------------------------------------------------------------
```

16.6 Deliberately deferred

```
Per §14.4: workload authn/authz and network/egress policy (SCOPE-1); control-plane
HA and full sharding (SCOPE-2); at-rest encryption of private deltas (SCOPE-3).
A deployment relying on LLIFS alone for isolation MUST run trusted workloads in an
isolated cluster until SCOPE-1 is supplied.

------------------------------------------------------------

End of specification.

------------------------------------------------------------
```

17. Glossary

```
Canonical terms (the only terms this document uses for stored state and lifecycle):

  block                 2 MiB Terrapin fetch+verify unit. (Was: "chunk".)
  content object        Terrapin object = one file's content bytes; filesystem
                        dedup unit. (Was: "artifact"; "chunk map" for files.)
  base rootfs           Shared read-only filesystem base (LLT1 tree + content
                        objects via LLAY1).
  base memory snapshot  Shared read-only 2 MiB-linear guest-memory image. (Was:
                        "golden memory" as a distinct object; "golden" is now an
                        informal adjective only.)
  base descriptor       Signed identity binding OCI image digest + base rootfs +
                        base memory + runtime-state + memory layout + sandbox pin
                        + compatibility matrix + assurance mode + runtime-state
                        policy, as one unit (§2.5, DIGEST-BIND-3). (Subsumes the
                        earlier "checkpoint manifest".)
  overlay               Live per-agent writable layer (dirty pages + fs upper);
                        ephemeral, never in shared CAS. (Was: "divergence".)
  delta                 Persisted content-addressed snapshot of an overlay, in three
                        parts: memory delta = a page MAP naming each diverged page's
                        source (this-delta/ancestor/base/zero, §15 LLMD1); filesystem
                        delta = overlay-delta object (content-object refs + namespace/
                        metadata ops + whiteouts, §15 LLFD1); runtime delta = opaque
                        runtime-native non-memory state (§15 LLRD1). (Was: "plane
                        manifest"; "chunk map" for memory.)
  State Root            Content-addressed identity of a persisted agent snapshot,
                        binding actor + base ref + memory delta + filesystem delta
                        + runtime delta + sandbox pin + workload identity + run mode
                        + producer + created + parent.
  actor                 The stable identity of an agent; survives across all of its
                        snapshots' State Roots. A fork allocates a NEW actor (§6.3).
  lineage               Parent-pointer DAG over State Roots; basis of forking.
  reset / hibernate / yield
                        RAM-reclamation mechanisms (§6): reset = restore base
                        with no delta; hibernate = base + stored delta; yield =
                        hibernate during a blocking upstream call.
  tier (local/external) Where a delta lives: local (node SSD) or external (object
                        storage). Orthogonal to the reclamation mechanism; maps to
                        the control plane's PAUSED (local) / SUSPENDED (external).
  cache domain          The maximum scope within which content-presence timing
                        leakage is acceptable; the isolation unit for dedup,
                        keying, warmth, and GC (§9).
  warmth / coverage     The fraction of an object/profile's required blocks (or
                        content objects, or base) already present on a node (§10).
  TTFE / TTR            Time-to-first-exec (cold start) / time-to-resume (warm
                        snapshot restore) (§7).
  profile               A weakly-trusted hint recording the working set faulted
                        earliest (startup profile = cold, resume profile = warm);
                        optimizes, never compels (§7.1). Keyed per PROFILE-3.
  runtime requirements  Trusted policy that COMPELS (mandatory prehydration,
                        fail-closed, expected-mutable handling); from signed
                        metadata/admission/operator config, never an untrusted
                        profile (§7.2).
  pack                  A transport envelope of canonical BlockRefs (startup pack =
                        cold, resume pack = warm), scatter-extracted into the CAS;
```

```
                        carries no Terrapin identity of its own (§7.3).
  bootstrap             The compact metadata record carrying all fetch + trust
                        inputs so a start needs no metadata/proof/attestation round
                        trip (startup/resume bootstrap, §7.4).
  coverage              The fraction of a required set (BlockRefs, or content
                        objects/base for warmth) already present on a node (§7.5,
                        §10).
  singleflight          The per-BlockRef state machine that collapses concurrent
                        reads/faults for one block into a single in-flight fetch
                        (§8.2).
  hedge                 A redundant fetch issued for a slow P0 request; first
                        VERIFIED response wins, losers cancel (§8.4).
  origin / mirror / peer / source
                        Fetch sources: origin = the authoritative registry; mirror
                        = a near-cluster pull-through cache/CDN; peer = another
                        node (untrusted for integrity, §8.6); source = any of these
                        chosen by source selection (§8.5).
  origin breaker        A per-registry/per-image circuit breaker + budget that
                        stops P0 hedges from stampeding a degraded origin (§8.4).
  lease                 A reference that pins an object alive in a cache domain;
                        types include runtime/startup/profile/peer/operator-pin/
                        cache-domain/materialization/pause/state-root (§9.2). Leases
                        are GC roots (§9.3).
  template/operator lease
                        A lease pinning shared base objects while a template is live,
                        so the base is not GC'd by per-agent churn (§9.3).
  epoch                 The version of a per-domain cache key; rotation bumps the
                        epoch and writes under the new key without invalidating
                        existing entries (§9.1).
  trusted-shared / private-domain
                        Cache keying modes: trusted-shared keys by the plain
                        BlockRef (cross-domain dedup, opt-in); private-domain keys by
                        HMAC(domainKey[epoch], BlockRef) so presence does not leak
                        across domains (§9.1).
  mark-sweep            The GC model: retain every object reachable from a root
                        (live lease or live State Root), sweep the rest (§9.3).
  logicalRootfsDigest   Terrapin digest of the canonical logical rootfs tree
                        (LLT1, §3); the supply-chain equivalence claim.
  layoutDigest          Terrapin digest of the canonical physical layout (LLAY1,
                        §4).

Supporting terms (structural objects, committed fields, and infrastructure,
not agent-state object classes, see §2.3):

  page                  The platform MMU page (4 KiB on x86-64); the unit of
                        per-agent copy-on-write memory sharing (§2.4, §5).
  CAS / node CAS        The node-local content-addressed store of verified blocks,
                        keyed per §9.
  imagefsd              The privileged node-local LLIFS daemon: resolves metadata,
                        maintains the CAS, fetches and verifies blocks, serves
                        reads (§8, §11).
  runtime-state blob    The non-memory sandbox state needed to resume (threads,
                        fds, namespaces, vCPU/sentry state); a structural object
                        committed by the base descriptor (§5).
  memory layout         The mapping from guest address space to base-memory-object
                        offsets (guest-address-linear); committed by the base
                        descriptor (§5).
  overlay-delta object  The canonical filesystem-delta encoding: content-object
                        refs + namespace/metadata ops + whiteouts (§2.3, §6, §15).
  startup-cohesion object
                        A per-image structural object that would pack hot
                        startup-critical small-file bytes for minimal startup block
                        count; RESERVED in v1 (packs are the v1 mechanism). If ever
                        defined (LLCOH1): no dedup/warmth, additive, never the sole
                        source of bytes (§4.4). Not a content object.
  materialization strictness
                        Policy for how much state must be present before a
                        lifecycle milestone, e.g. startup-only, readiness-
                        critical, full-before-ready/exec, lazy-risk-accepted
                        (§5, §7).
  STARTUP_READY         The materialization milestone at which startup-critical
                        metadata, proofs, and blocks are present so the workload
                        can begin executing (§7).
```

attestation envelope	The runtime-provenance signature over a State Root identifier (signer, trust root, subject, validation; §2.5).
sandbox pin	Per-architecture, content-addressed (external sha256) identity of the runtime assets that produced a snapshot; a committed field constraining restore placement (§2.5).
assurance mode	The base's supply-chain assurance level (asserted / derived-attested / node-derived; §3, §14); a committed field of the base descriptor.
workload identity	The canonical digest of the restore-visible workload fields (pause image, hostname, namespace flags, pod/container security context, and per container name/image/command/args/ env, working dir, run-as user, capabilities, mount/device topology, and resource limits; LLWI1 v2, §15.8); a committed field of a State Root (§6).
run mode	Which planes persist across a lifecycle event: clean, golden, persist-rootfs, or persist-rootfs+memory; a committed field of a State Root (§6.6). NB "golden" is a run-mode name (boot from a shared base memory snapshot), NOT an object class; "golden memory" as an object is retired.
producer	The identity recorded in a State Root as having produced it. The attestation-envelope signer MUST equal the producer or be policy-authorized to attest for it (§2.5).
base	Shorthand for a base descriptor and the shared, read-only state it binds (base rootfs + base memory snapshot).
node trust boundary	The set of components (kernel, runtime, imagefsd, trust roots) trusted on a node; the scope within which runtime attestation is issued and validated (§2.5, §14).
node score	The scheduler's per-node placement score combining cache-domain-scoped warmth minus node pressure (§10.2).
resume affinity	The scheduler preference for a node already holding an actor's local delta and/or warm base, for fast resume; advisory only (§10.3).
seed node	A node chosen to receive a prewarmed bootstrap/pack for a rollout or fork, from which others fan out (§10.4).
prewarm	Proactively fetching/verifying blocks (bootstrap, pack, base) onto nodes before they are needed (§7.4, §10.4).
mmap/exec-critical	Files/segments required for execve and dynamic linking (and selected mmap paths) that SHOULD be present before exec (§7, §10).
runtime adapter	A per-runtime layer (gVisor, Firecracker) that sources verified bytes from the CAS without changing internal identity or the verify-before-expose invariant (§11).
Sentry / gofer / lisafs	gVisor's user-space guest kernel (Sentry); the file-serving process it talks to (gofer) over the lisafs protocol; LLIFS serves the read-only lower through a CAS-backed gofer (§11.2).
MemoryFile	gVisor's guest-memory backing; the shared copy-on-write base capability (GVISOR-3) extends it to map a shared verified base file (§11.2).
platform mode	The gVisor execution platform (e.g. ptrace/systrap/KVM); a compatibility-matrix field (§11.4).
compatibility matrix	The base descriptor's record of what a restore requires (runtime+version, platform mode, arch+page size, CPU features, checkpoint format, ABI, seccomp/cgroup/ns); a mismatch fails closed (§11.4).
native backend	A later page-cache-integrated FS backend (EROFS/composefs/ fs-verity/virtiofs) that preserves Terrapin identity and cache-domain policy (§11.5).
BlockRef	The canonical address of a block: object-digest / level / blockIndex (level 0 = data, 1+ = hash-file). The unit of coverage, packs, and fetch scheduling.
P0	The highest fetch priority class: a synchronous foreground read or page fault that the workload is blocked on; P0 preempts background hydration (§8).
restore working set	The access-ordered set of base memory snapshot blocks to prehydrate before resume; the memory-plane analog of the startup pack (§5.5, §7).
base version	An opaque identifier for one captured base (a base descriptor and its objects); base versions are few and each is stored as its own object (§5).
memory profile	The profile keying a restore working set to a base version (analogous to a startup profile, §7).

```
  HAZARDS_CLEARED       The memory-restore lifecycle milestone at which RHAZARD-1..7
                        have been refreshed/invalidated so a restored agent may
                        serve (§5.6, §5.7).

Retired terms (do not use): chunk, chunk map, artifact, plane manifest, opaque
checkpoint, checkpoint plane, checkpoint manifest, golden memory (as an object),
state-root-sha256 (as a separate scheme), "co-design"/team references for runtime
changes (state required runtime capabilities technically, §11).
```